

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

BÁO CÁO TỔNG KẾT
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

PHÁT TRIỂN HỆ THỐNG HỖ TRỢ
LẬP KẾ HOẠCH HỌC TẬP CHO SINH VIÊN

Mã số đề tài: 1187

Tp. Hồ Chí Minh, tháng 03/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

BÁO CÁO TỔNG KẾT
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN

PHÁT TRIỂN HỆ THỐNG HỖ TRỢ
LẬP KẾ HOẠCH HỌC TẬP CHO SINH VIÊN

Mã số đề tài: 1187

Chủ nhiệm đề tài: Nguyễn Phan Thiên Phát

Khoa: Đào tạo đặc biệt

Các thành viên: Võ Châu Anh Kiệt; Từ Ngọc Anh Khoa;

Nguyễn Quốc Bảo; Nguyễn Trung Kiên

Giảng viên hướng dẫn: ThS. Nguyễn Văn Bảy

Tp. Hồ Chí Minh, tháng 03/2026

MỤC LỤC

Mục lục	i
Danh mục bảng	iv
Danh mục hình	v
Lời mở đầu	vii
1 CƠ SỞ LÝ THUYẾT CÔNG NGHỆ VÀ NỀN TẢNG	1
1.1 Quản Lý thời gian trong học tập	1
1.2 Phương pháp Kanban trong quản lý công việc	3
1.3 Phương pháp Pomodoro trong quản lý thời gian học tập	6
1.4 Phương pháp nghiên cứu công việc	7
2 PHƯƠNG PHÁP NGHIÊN CỨU VÀ KHẢO SÁT YÊU CẦU HỆ THỐNG	10
2.1 Phương pháp nghiên cứu	10
2.2 Phân tích hệ thống tham chiếu	11
2.3 Phân tích và đặc tả yêu cầu hệ thống	15
3 THIẾT KẾ KIẾN TRÚC HỆ THỐNG VÀ THUẬT TOÁN	18

3.1	Kiến trúc tổng thể của hệ thống	19
3.2	Thiết kế cấu trúc cơ sở dữ liệu	20
3.3	Thiết kế hệ thống API	26
3.4	Thuật toán phân loại độ ưu tiên dựa trên ranh giới thời gian .	29
3.5	Thuật toán đánh giá tiến độ dựa trên thời lượng (Time-Weighted Progress Algorithm)	33
3.6	Cơ chế cập nhật tự động bằng tiến trình chạy ngầm (Cron Job)	34
3.7	Thiết kế giao diện người dùng (Frontend Architecture)	36
4	KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	38
4.1	Môi trường và công cụ triển khai hệ thống	38
4.2	Kết quả triển khai các phân hệ chức năng	40
4.3	Đánh giá thực nghiệm kiến trúc và thuật toán	44
4.4	Hạn chế và hướng phát triển trong tương lai	46
4.5	Kết luận và hướng phát triển	47
4.5.1	Kết luận	47
4.5.2	Hướng phát triển	48
	Hướng dẫn sử dụng hệ thống Smart Study Planner	50
1	Khởi tạo và đăng nhập hệ thống	50
2	Hướng dẫn phân hệ các chức năng chính	52
2.1	Phân hệ quản lý môn học (Subjects)	52
2.2	Phân hệ quản lý lịch học (Calendar)	54
2.3	Phân hệ quản lý công việc (Task)	56
2.4	Phân hệ hỗ trợ tập trung (Pomodoro timer)	58
2.5	Phân hệ hồ sơ và cài đặt (Profile Setting)	59
2.6	Bảng điều khiển trung tâm (DashBoard)	60

DANH MỤC BẢNG

2.1	Bảng phân tích và so sánh tính năng giữa các nền tảng quản lý thời gian hiện nay.	14
3.1	Chi tiết thực thể User (Người dùng)	23
3.2	Chi tiết thực thể Subject (Môn học)	24
3.3	Chi tiết thực thể Task (Nhiệm vụ học tập)	25
3.4	Chi tiết thực thể Student Session (Phiên học)	26
3.5	Danh sách các RESTful API cốt lõi của hệ thống	27
4.1	Cấu hình môi trường thực nghiệm và phát triển hệ thống . . .	39
4.2	Đánh giá thời gian phản hồi của các API tiêu biểu	46

DANH MỤC HÌNH VÀ ĐỒ THỊ

1.1	Ma trận Eisenhower phân loại công việc theo mức độ quan trọng và khẩn cấp	3
1.2	Mô hình quản lý công việc trực quan bằng bảng Kanban . . .	5
1.3	Chu kỳ làm việc và nghỉ ngơi theo kỹ thuật pomodoro	7
1.4	Mô hình mô phỏng cơ chế phân loại ưu tiên tự động dựa trên thời gian	9
2.1	Use case mô tả các tương tác của sinh viên với hệ thống . . .	15
3.1	Sơ đồ thực thể - Quan hệ (ERD) của cơ sở dữ liệu	22
3.2	Lưu đồ thuật toán tính toán mức độ ưu tiên dựa trên ranh giới thời gian	32
3.3	Sơ đồ tuần tự (Sequence Diagram) mô tả luồng	35
4.1	Giao diện đăng nhập và xác thực người dùng tích hợp kiểm duyệt lỗi trực tiếp	41
4.2	Giao diện bảng điều khiển (DashBoard) tổng quan tiến độ học tập	42
4.3	Quản lý vòng đời bài tập bằng mô hình bảng Kanban	43
4.4	Giao diện bộ đếm giờ Pomodoro và biểu đồ thống kê nỗ lực học tập	44

1.1	Giao diện đăng nhập và cài đặt PWA của hệ thống	51
2.1	Giao diện quản lý môn học và tính năng cảnh báo trùng lặp .	53
2.2	Giao diện lưới thời khóa biểu và tính năng chống trùng lịch . .	55
2.3	Giao diện quản lý công việc đa góc nhìn	57
2.4	Đồng hồ đếm ngược Pomodoro tích hợp đồng bộ trạng thái công việc	59
2.5	Giao diện hồ sơ cá nhân và theo dõi tiến độ mục tiêu	60
2.6	Bảng điều khiển trung tâm thể hiện tổng quan tiến độ	61

LỜI MỞ ĐẦU

Trong nghiên cứu này, nhóm đề xuất xây dựng hệ thống Smart Study Planner, một ứng dụng web hỗ trợ sinh viên quản lý và sắp xếp các nhiệm vụ học tập hiệu quả hơn. Hệ thống tích hợp mô hình quản lý công việc Kanban, kỹ thuật Pomodoro nhằm tăng khả năng tập trung khi học tập, cùng với thuật toán Auto-Priority được xây dựng dựa trên phương pháp Weighted Shortest Job First (WSJF) để tự động xác định mức độ ưu tiên của nhiệm vụ dựa trên thời gian còn lại trước hạn nộp. Hệ thống được phát triển theo kiến trúc Client–Server sử dụng Node.js và cơ chế Cron Job để tự động cập nhật mức độ ưu tiên của các nhiệm vụ.

Kết quả thử nghiệm bước đầu cho thấy hệ thống giúp sinh viên theo dõi tiến độ học tập tốt hơn, giảm thời gian quyết định thứ tự công việc và hỗ trợ tổ chức kế hoạch học tập một cách hiệu quả hơn.

Khi chuyển từ môi trường phổ thông sang đại học, sinh viên thường phải thích nghi với nhiều thay đổi trong phương pháp học tập. Quá trình học ở bậc đại học đòi hỏi mức độ tự chủ cao hơn, khả năng tự định hướng trong học tập, đồng thời phải xử lý khối lượng kiến thức lớn cùng với nhiều hoạt động học thuật như bài tập học phần (assignment), đồ án, dự án nhóm và các hoạt động ngoại khóa. Tuy nhiên, trong thực tế, kỹ năng quản lý thời gian và tổ chức công việc học tập vẫn còn là một hạn chế phổ biến đối với

nhiều sinh viên.

Các nghiên cứu trong lĩnh vực tâm lý học giáo dục cho thấy hiện tượng trì hoãn trong học tập (procrastination) xuất hiện với tần suất khá cao trong cộng đồng sinh viên. Thống kê từ một số công trình nghiên cứu chỉ ra rằng khoảng 70% – 80% sinh viên đại học thường xuyên trì hoãn việc hoàn thành các nhiệm vụ học tập [1]. Việc thiếu kỹ năng lập kế hoạch và phân bổ thời gian hợp lý không chỉ dẫn đến tình trạng nộp bài trễ hạn (miss deadline), mà còn làm gia tăng áp lực tâm lý kéo dài. Điều này có thể gây ảnh hưởng tiêu cực đến sức khỏe tinh thần cũng như kết quả học tập của sinh viên trong dài hạn [2].

Trong bối cảnh đó, nhiều công cụ hỗ trợ quản lý công việc và ghi chú đã được phát triển và sử dụng rộng rãi, tiêu biểu như Notion, Trello hay Google Calendar. Những nền tảng này cung cấp các chức năng cơ bản như tạo danh sách công việc, theo dõi tiến độ hoặc quản lý lịch trình cá nhân. Tuy nhiên, phần lớn các hệ thống này được thiết kế với mục tiêu phục vụ nhiều lĩnh vực khác nhau và chưa thực sự tối ưu cho nhu cầu học tập của sinh viên. Một hạn chế đáng chú ý của các công cụ này là tính chất tương đối thụ động: người dùng vẫn phải tự thiết lập cấu trúc quản lý công việc và tự đánh giá mức độ ưu tiên của từng nhiệm vụ.

Đối với những sinh viên chưa có nhiều kinh nghiệm trong việc lập kế hoạch học tập, việc phải xử lý một danh sách dài các nhiệm vụ chưa được phân loại rõ ràng có thể gây ra hiện tượng “mệt mỏi khi ra quyết định” (decision fatigue) [3]. Khi phải liên tục đưa ra nhiều lựa chọn trong thời gian ngắn, khả năng ra quyết định hiệu quả có xu hướng suy giảm, khiến quá trình quản lý công việc trở nên tốn thời gian và kém hiệu quả hơn. Bên cạnh đó, việc sử dụng nhiều ứng dụng khác nhau để quản lý lịch học, theo dõi tiến độ và duy trì sự tập trung cũng dễ dẫn đến tình trạng phân tán trong quá trình làm

việc.

Xuất phát từ những hạn chế trên, đề tài này đề xuất phát triển một hệ thống mang tên Smart Study Planner, được triển khai dưới dạng ứng dụng web nhằm hỗ trợ sinh viên trong việc quản lý công việc học tập và tối ưu hóa quá trình lập kế hoạch cá nhân. Hệ thống được xây dựng với mục tiêu kế thừa các ưu điểm của những công cụ quản lý hiện có, đồng thời khắc phục một số điểm hạn chế để phù hợp hơn với bối cảnh học tập của sinh viên.

Đề tài tập trung vào ba đóng góp chính trong việc ứng dụng công nghệ vào quản lý học vụ. Thứ nhất, hệ thống tích hợp các phương pháp quản lý năng suất phổ biến như bảng tiến độ Kanban và kỹ thuật Pomodoro vào cùng một nền tảng. Việc kết hợp các phương pháp này giúp sinh viên dễ dàng trực quan hóa tiến độ công việc, đồng thời duy trì sự tập trung trong quá trình học tập mà không cần phải sử dụng nhiều ứng dụng khác nhau, từ đó giảm thiểu hiện tượng chuyển đổi ngữ cảnh (context switching).

Thứ hai, đề tài thiết kế và triển khai một thuật toán tự động xác định mức độ ưu tiên của công việc (Auto-Priority) dựa trên ý tưởng của phương pháp Weighted Shortest Job First (WSJF). Thuật toán này cho phép hệ thống tự động đánh giá mức độ khẩn cấp của các nhiệm vụ học tập dựa trên khoảng thời gian còn lại trước hạn nộp, từ đó hỗ trợ sinh viên xác định thứ tự thực hiện công việc một cách hợp lý và giảm bớt yếu tố cảm tính trong quá trình ra quyết định. Thứ ba, hệ thống được triển khai với cơ chế cập nhật tự động thông qua một tiến trình chạy nền (Cron Job) trên máy chủ Node.js. Cơ chế này giúp hệ thống định kỳ cập nhật lại mức độ ưu tiên của các nhiệm vụ theo thời gian thực, đảm bảo rằng danh sách công việc luôn phản ánh đúng mức độ khẩn cấp hiện tại mà không cần sự can thiệp thủ công từ phía người dùng. Nhờ đó, hệ thống có thể hoạt động tương tự như một trợ lý hỗ trợ lập kế hoạch học tập cá nhân.

Phần còn lại của báo cáo được tổ chức như sau:

- Chương 1 trình bày các cơ sở lý thuyết liên quan đến quản lý thời gian, các phương pháp nâng cao hiệu quả học tập như Pomodoro và Kanban, cùng với các công nghệ nền tảng được sử dụng trong hệ thống.
- Chương 2 mô tả phương pháp nghiên cứu, quá trình khảo sát và phân tích yêu cầu hệ thống.
- Chương 3 tập trung vào thiết kế kiến trúc hệ thống theo mô hình Client-Server, thiết kế cơ sở dữ liệu và phân tích chi tiết các thành phần chức năng của thuật toán Auto-Priority.
- Chương 4 trình bày quá trình triển khai hệ thống, kết quả thực nghiệm và đánh giá hiệu quả sử dụng trong thực tế.
- Cuối cùng, phần Kết luận tổng kết các kết quả đạt được, đề xuất hướng phát triển trong tương lai và hướng dẫn sử dụng hệ thống.

Chương 1

CƠ SỞ LÝ THUYẾT CÔNG NGHỆ VÀ NỀN TẢNG

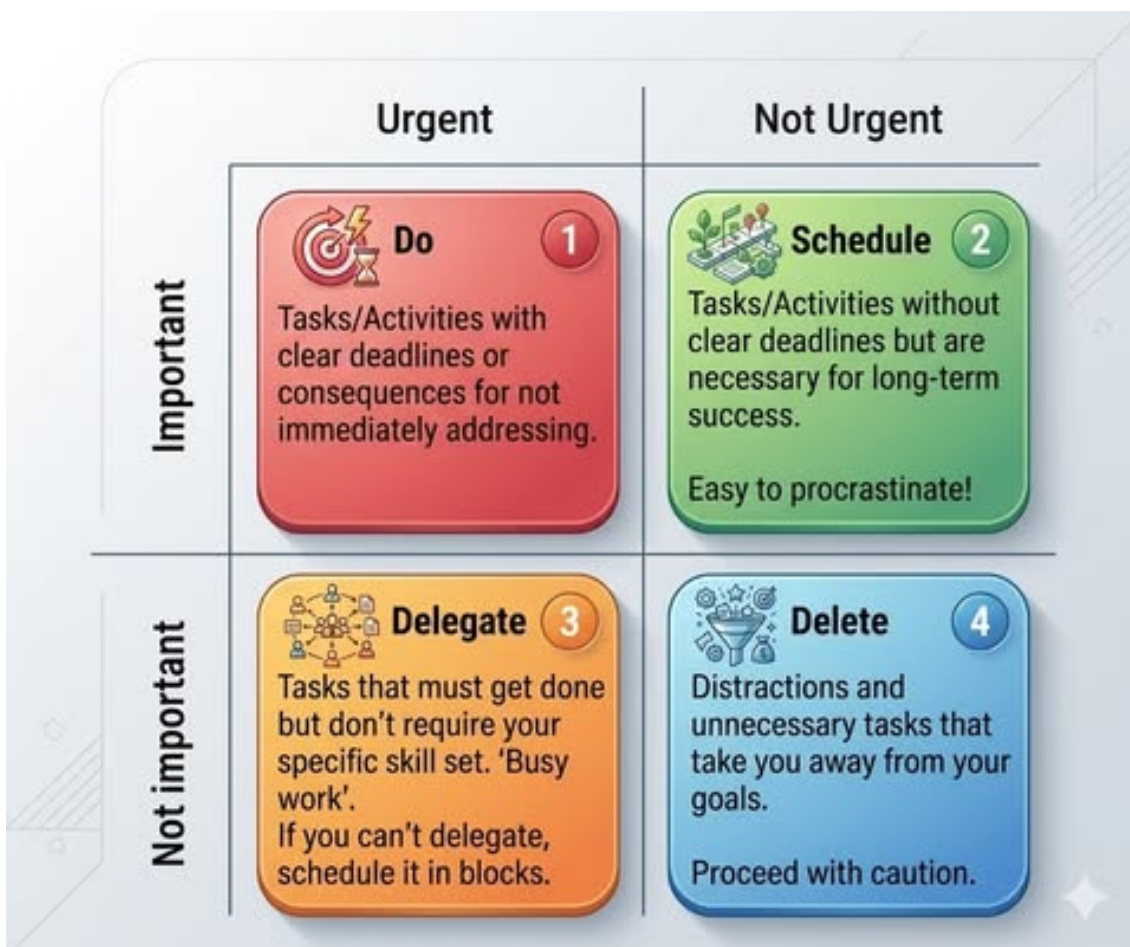
Chương này trình bày các nền tảng lý thuyết liên quan đến quản lý thời gian trong học tập, các phương pháp quản lý công việc và cơ chế ưu tiên nhiệm vụ được sử dụng trong hệ thống. Bên cạnh đó, chương cũng giới thiệu các công nghệ và framework được lựa chọn để phát triển hệ thống Smart Study Planner.

1.1 Quản Lý thời gian trong học tập

Quản lý thời gian được xem là một trong những kỹ năng quan trọng nhất giúp nâng cao hiệu quả học tập và làm việc của sinh viên. Trong môi trường đại học, sinh viên thường phải xử lý nhiều nhiệm vụ song song như bài tập cá nhân, dự án nhóm, lịch học, hoạt động ngoại khóa và các trách nhiệm cá nhân khác. Khi thiếu kỹ năng tổ chức thời gian hợp lý, người học rất dễ rơi vào trạng thái quá tải thông tin (cognitive overload), từ đó dẫn đến hiện tượng trì hoãn (procrastination), bỏ lỡ thời hạn nộp bài hoặc suy giảm hiệu suất học tập [4]. Các nghiên cứu trong lĩnh vực tâm lý học giáo dục chỉ ra rằng khả năng quản lý thời gian có mối liên hệ chặt chẽ với kết quả học

tập và mức độ tự chủ của sinh viên. Theo lý thuyết Học tập tự điều chỉnh (Self-Regulated Learning – SRL), người học hiệu quả thường có khả năng tự đặt mục tiêu, giám sát tiến độ và điều chỉnh chiến lược học tập của mình [5]. Theo mô hình này, quá trình học tập thường diễn ra theo một chu trình gồm ba giai đoạn chính:

- Lập kế hoạch (Planning): xác định mục tiêu và xây dựng chiến lược thực hiện.
- Thực thi (Performance): duy trì sự tập trung và theo dõi tiến trình học tập.
- Phản tư (Reflection): đánh giá kết quả và điều chỉnh phương pháp học tập. Những hệ thống hỗ trợ lập kế hoạch học tập hiện đại được thiết kế nhằm hỗ trợ chu trình này, giúp người dùng thiết lập mục tiêu rõ ràng, phân chia nhiệm vụ thành các bước nhỏ và theo dõi tiến độ thực hiện một cách trực quan. Theo một số nghiên cứu gần đây, việc sử dụng các công cụ lập kế hoạch học tập kỹ thuật số có thể cải thiện đáng kể khả năng tự quản lý thời gian và mức độ hoàn thành nhiệm vụ của sinh viên [6]. Tuy nhiên, trong thực tế, nhiều ứng dụng quản lý công việc hiện nay chỉ dừng lại ở chức năng tạo danh sách công việc (to-do list) đơn giản. Để nâng cao hiệu quả sử dụng, các hệ thống này cần được thiết kế dựa trên các nguyên lý khoa học về năng suất, đồng thời tích hợp cơ chế tự động phân tích mức độ ưu tiên và hỗ trợ người dùng duy trì sự tập trung trong quá trình học tập.



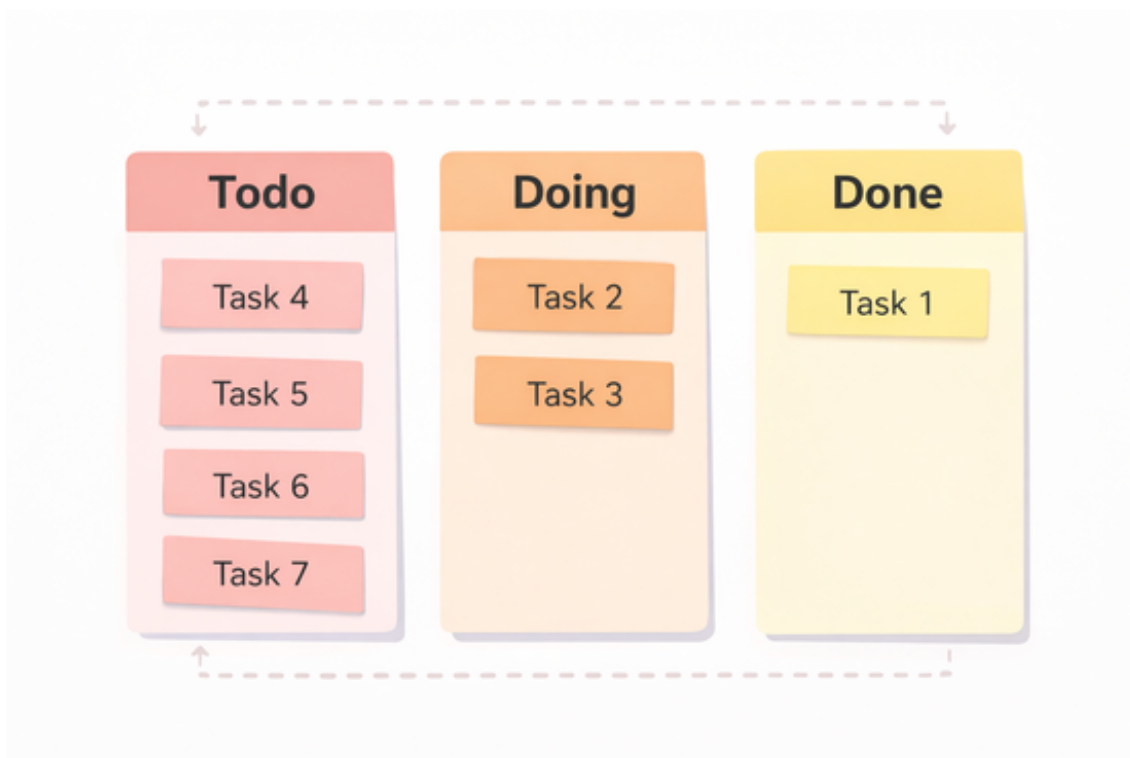
Hình 1.1: Ma trận Eisenhower phân loại công việc theo mức độ quan trọng và khẩn cấp

1.2 Phương pháp Kanban trong quản lý công việc

Kanban là một phương pháp quản lý công việc trực quan có nguồn gốc từ hệ thống sản xuất tinh gọn (Lean Manufacturing) của Toyota. Ban đầu phương pháp này được sử dụng để tối ưu quy trình sản xuất, sau đó được áp dụng rộng rãi trong quản lý dự án phần mềm và quản lý công việc cá nhân [7]. Thuật ngữ “Kanban” trong tiếng Nhật có nghĩa là “bảng tín hiệu” hoặc

“bảng thông tin trực quan”. Ban đầu, phương pháp này được sử dụng để tối ưu hóa quy trình sản xuất, nhưng ngày nay đã được ứng dụng rộng rãi trong quản lý dự án phần mềm theo mô hình Agile cũng như trong quản lý năng suất cá nhân [8]. Nguyên lý cốt lõi của Kanban dựa trên hai yếu tố chính: trực quan hóa công việc (visualization) và giới hạn khối lượng công việc đang thực hiện (Work-In-Progress – WIP). Công việc được biểu diễn dưới dạng các thẻ nhiệm vụ và được sắp xếp trên một bảng Kanban gồm nhiều cột, mỗi cột đại diện cho một giai đoạn trong quy trình thực hiện. Trong mô hình Kanban cơ bản áp dụng cho cá nhân, các nhiệm vụ thường được tổ chức thành ba trạng thái chính:

- To Do (Cần thực hiện): danh sách các nhiệm vụ đã được lên kế hoạch nhưng chưa bắt đầu.
- In Progress (Đang thực hiện): các nhiệm vụ đang được xử lý. Việc giới hạn số lượng thẻ trong cột này giúp tránh tình trạng đa nhiệm (multitasking), vốn được chứng minh là làm giảm hiệu suất làm việc.
- Done (Hoàn thành): các nhiệm vụ đã hoàn tất.

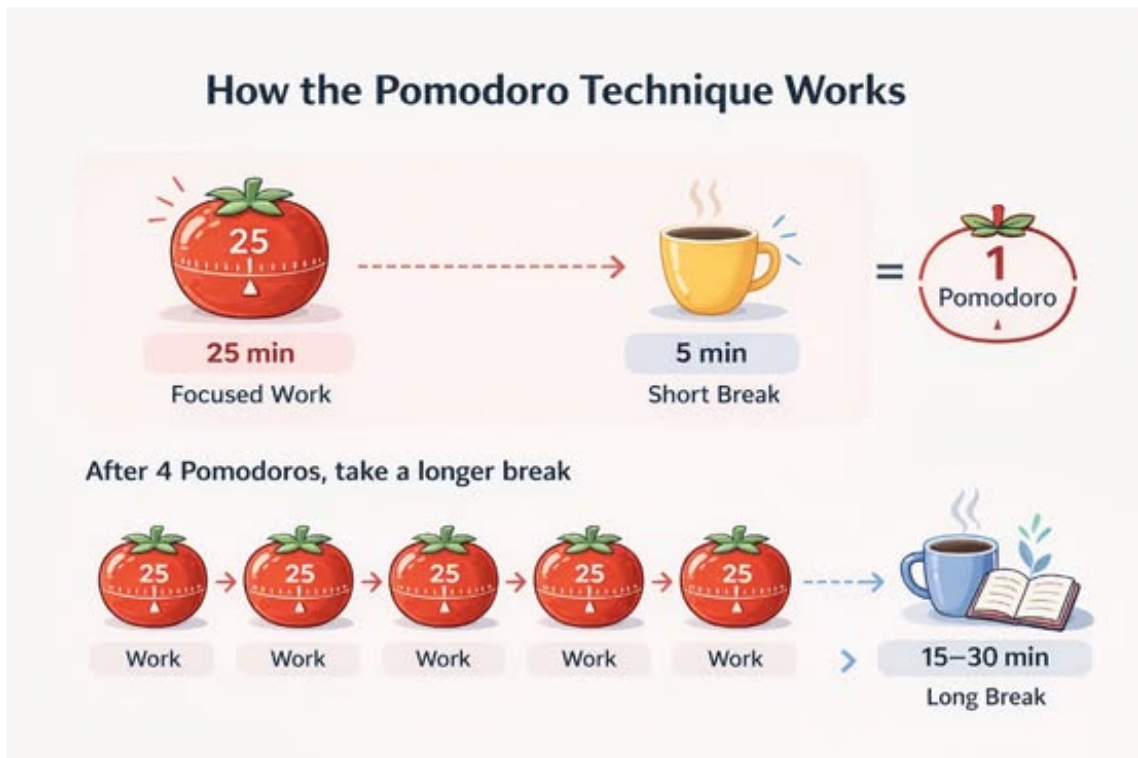


Hình 1.2: Mô hình quản lý công việc trực quan bằng bảng Kanban

Ngoài giá trị về mặt tổ chức công việc, Kanban còn tạo ra hiệu ứng tích cực về tâm lý. Khi người dùng thường cảm thấy hài lòng khi hoàn thành một nhiệm vụ và chuyển thẻ sang cột “Done”. Điều này tạo ra động lực tiếp tục hoàn thành các nhiệm vụ tiếp theo. Hiện tượng này có liên hệ với hiệu ứng Zeigarnik, trong đó con người có xu hướng ghi nhớ và quan tâm nhiều hơn đến những nhiệm vụ chưa hoàn thành [9]. Trong hệ thống Smart Study Planner, bảng Kanban được sử dụng làm giao diện trung tâm để người dùng tạo nhiệm vụ, theo dõi tiến độ và cập nhật trạng thái công việc hàng ngày.

1.3 Phương pháp Pomodoro trong quản lý thời gian học tập

Kỹ thuật Pomodoro là một phương pháp quản lý thời gian do Francesco Cirillo phát triển vào cuối thập niên 1980 nhằm cải thiện khả năng tập trung khi làm việc hoặc học tập. Tên gọi “Pomodoro” bắt nguồn từ chiếc đồng hồ đếm giờ hình quả cà chua mà tác giả sử dụng khi còn là sinh viên. Pomodoro là một kỹ thuật quản lý thời gian phổ biến, trong đó công việc được chia thành các khoảng thời gian tập trung ngắn xen kẽ với các khoảng nghỉ. Phương pháp này giúp giảm mệt mỏi nhận thức và duy trì sự tập trung khi thực hiện các nhiệm vụ học tập. Các nghiên cứu gần đây cho thấy việc áp dụng Pomodoro có thể hỗ trợ sinh viên cải thiện khả năng tập trung và quản lý thời gian trong quá trình học tập trực tuyến [10]. Một chu kỳ Pomodoro điển hình bao gồm 25 phút tập trung làm việc và 5 phút nghỉ ngắn. Sau bốn chu kỳ liên tiếp, người dùng sẽ có một khoảng nghỉ dài từ 15–30 phút.



Hình 1.3: Chu kỳ làm việc và nghỉ ngơi theo kỹ thuật pomodoro

Nhiều nghiên cứu gần đây cho thấy phương pháp Pomodoro có thể cải thiện đáng kể mức độ tập trung và giảm sự phân tâm khi học tập trực tuyến [11]. Ngoài ra, việc chia nhỏ công việc thành các khoảng thời gian ngắn còn giúp giảm bớt rào cản tâm lý khi bắt đầu nhiệm vụ, từ đó hạn chế tình trạng trì hoãn. Trong hệ thống Smart Study Planner, bộ đếm Pomodoro được tích hợp trực tiếp vào giao diện học tập. Nhờ vậy, người dùng có thể áp dụng phương pháp này mà không cần chuyển sang các ứng dụng khác, qua đó hạn chế sự gián đoạn và xao nhãng trong quá trình học tập.

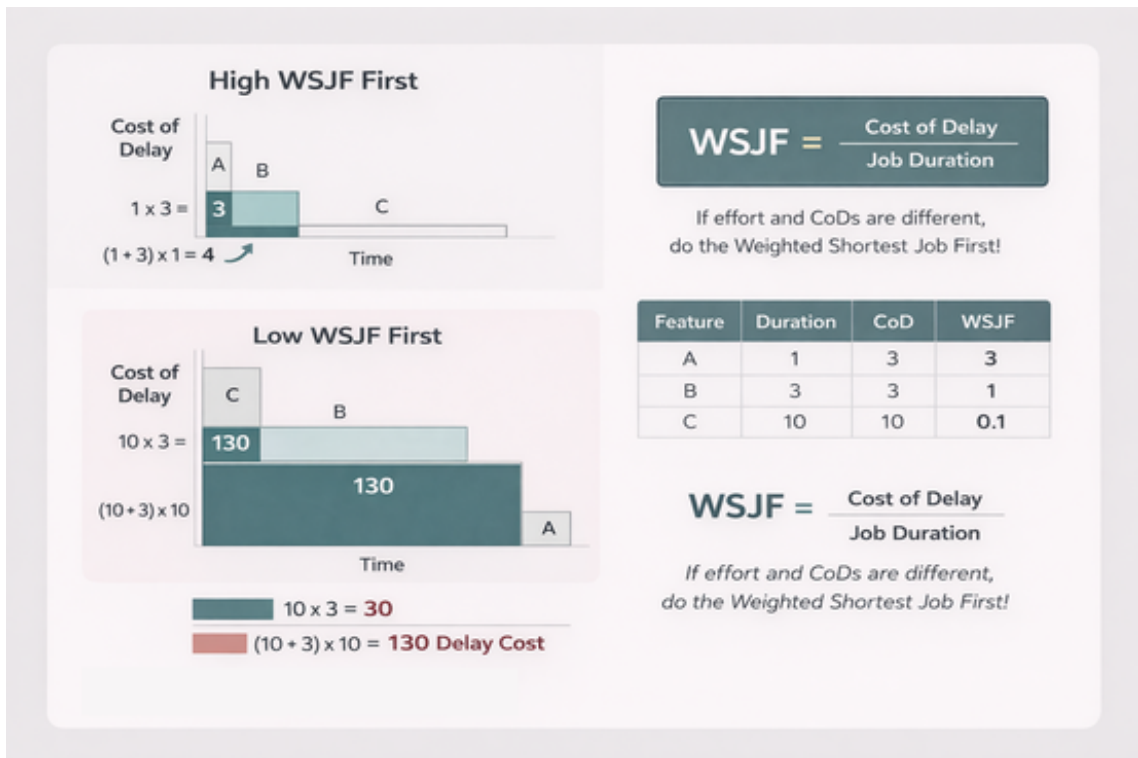
1.4 Phương pháp nghiên cứu công việc

Trong quá trình quản lý nhiệm vụ học tập, việc xác định mức độ ưu tiên của từng bài tập là yếu tố mang tính quyết định. Khi phải xử lý nhiều

deadline từ các môn học khác nhau, sinh viên thường đánh giá mức độ khẩn cấp của công việc dựa trên cảm tính. Điều này có thể dẫn đến việc dành thời gian cho những nhiệm vụ dễ thực hiện thay vì những nhiệm vụ quan trọng sắp đến hạn.[12]

Để giải quyết vấn đề này, hệ thống Smart Study Planner áp dụng ý tưởng từ thuật toán Weighted Shortest Job First (WSJF). Đây là một phương pháp được sử dụng trong khung làm việc Scaled Agile Framework (SAFe) nhằm xác định mức độ ưu tiên của các nhiệm vụ dựa trên tỷ lệ giữa chi phí trì hoãn (Cost of Delay) và quy mô công việc (Job Size) [13].

Trong bối cảnh học tập, chi phí trì hoãn thường liên quan đến việc bị trừ điểm hoặc mất cơ hội hoàn thành bài tập đúng hạn. Vì vậy, hệ thống đã đơn giản hóa mô hình WSJF bằng cách tập trung vào yếu tố thời gian còn lại trước deadline.



Hình 1.4: Mô hình mô phỏng cơ chế phân loại ưu tiên tự động dựa trên thời gian

Theo cơ chế này, các nhiệm vụ có thời gian còn lại càng ngắn sẽ được hệ thống tự động gán mức độ ưu tiên cao hơn. Điều này giúp người dùng nhanh chóng nhận biết những nhiệm vụ quan trọng cần xử lý trước, từ đó hạn chế tình trạng bỏ lỡ deadline.

Chương 2

PHƯƠNG PHÁP NGHIÊN CỨU VÀ KHẢO SÁT YÊU CẦU HỆ THỐNG

Chương này trình bày phương pháp nghiên cứu được sử dụng trong quá trình xây dựng hệ thống Smart Study Planne. Nội dung tập trung vào việc tổng hợp các công trình nghiên cứu liên quan, khảo sát thực tế hành vi quản lý thời gian của sinh viên, đồng thời phân tích các hệ thống quản lý công việc phổ biến hiện nay. Từ những cơ sở lý luận và dữ liệu thu thập được, nhóm nghiên cứu tiến hành xác định các yêu cầu chức năng cốt lõi cũng như đề xuất kiến trúc tổng thể cho hệ thống.

2.1 Phương pháp nghiên cứu

Để đảm bảo hệ thống được xây dựng dựa trên cả cơ sở lý thuyết và nhu cầu thực tế, nghiên cứu này sử dụng phương pháp nghiên cứu hỗn hợp (Mixed Methods Research). Trước hết, phương pháp phân tích dữ liệu thứ cấp (Secondary Data Analysis) được sử dụng nhằm tổng hợp các kết quả nghiên cứu từ các tạp chí khoa học và bài báo học thuật trong lĩnh vực tâm lý học giáo dục và quản lý thời gian học tập [14]. Việc phân tích các nguồn tài liệu này giúp nhóm nghiên cứu hiểu rõ hơn về các yếu tố ảnh hưởng đến

hành vi trì hoãn học tập của sinh viên cũng như vai trò của các công cụ hỗ trợ lập kế hoạch học tập [15].

Phương pháp phân tích hệ thống so sánh (Comparative System Analysis) đối với một số nền tảng quản lý công việc phổ biến như Google Calendar, Trello và Notion. Thông qua việc phân tích các yếu tố liên quan đến Trải nghiệm người dùng (User Experience – UX) và Tương tác Người – Máy (Human–Computer Interaction – HCI), nhóm nghiên cứu xác định được những hạn chế tồn tại trong các công cụ hiện tại, từ đó đề xuất hướng cải tiến cho hệ thống mới. Kết quả của quá trình nghiên cứu này được sử dụng làm cơ sở để tiến hành phân tích yêu cầu hệ thống và thiết kế kiến trúc phần mềm, trước khi bước sang giai đoạn phát triển theo mô hình Agile.

2.2 Phân tích hệ thống tham chiếu

Khó khăn trong việc quản lý thời gian học tập của sinh viên là một vấn đề đã được đề cập trong nhiều nghiên cứu học thuật. Theo nghiên cứu của Fentaw và cộng sự (2022), môi trường đại học với khối lượng kiến thức lớn và mức độ tự chủ cao khiến khoảng 70–80% sinh viên thường xuyên rơi vào tình trạng trì hoãn học tập [5]. Hiện tượng này có thể dẫn đến nhiều hệ quả tiêu cực như giảm hiệu suất học tập, tăng mức độ căng thẳng và làm suy giảm động lực học tập. Một trong những nguyên nhân sâu xa của hiện tượng này được Arnold và cộng sự (2023) lý giải thông qua tình trạng "Quá tải thông tin" (Information Overload) và "Mệt mỏi tinh thần" (Mental Fatigue) [6]. Trong môi trường học tập hiện đại, khi sinh viên phải đối mặt với một danh sách dài các bài tập, đề án và lịch thi chồng chéo cùng lúc, năng lượng nhận thức của não bộ nhanh chóng bị bào mòn. Sự quá tải thông tin này làm suy giảm khả năng tổ chức và phân cấp công việc, dẫn đến việc sinh viên có

xu hướng bị ngợp, từ đó trì hoãn hoặc né tránh bắt tay vào các nhiệm vụ phức tạp. Điều này khẳng định tính cấp thiết phải có một hệ thống tự động hóa việc sắp xếp độ ưu tiên để giảm tải gánh nặng nhận thức cho sinh viên. Trong quá trình quản lý học tập, nhiều sinh viên lựa chọn sử dụng các công cụ kỹ thuật số để hỗ trợ tổ chức thời gian. Tuy nhiên, mỗi nền tảng hiện nay đều tồn tại những hạn chế nhất định khi áp dụng trong môi trường học tập: Google Calendar là công cụ phổ biến trong việc lập lịch theo mô hình time-blocking, cho phép người dùng quản lý các mốc thời gian một cách rõ ràng. Tuy nhiên, công cụ này chủ yếu được thiết kế như một lịch cá nhân và thiếu khả năng theo dõi chi tiết tiến độ của các nhiệm vụ phức tạp. Theo Chen và Lin (2022), việc phải sử dụng nhiều công cụ khác nhau để quản lý lịch và công việc có thể dẫn đến hiện tượng “tool fragmentation”, khiến người dùng phải liên tục chuyển đổi giữa các ứng dụng và làm giảm khả năng tập trung [4]. Trello là nền tảng quản lý công việc dựa trên mô hình bảng Kanban, cho phép trực quan hóa tiến độ nhiệm vụ khá hiệu quả. Tuy nhiên, hệ thống này vẫn phụ thuộc nhiều vào việc người dùng tự quản lý và cập nhật trạng thái công việc. Khi số lượng nhiệm vụ tăng lên, việc cập nhật trạng thái thủ công có thể trở nên bất tiện khi số lượng nhiệm vụ tăng lên, khiến người dùng dần bỏ qua việc sử dụng công cụ [5]. Notion là một nền tảng có khả năng tùy biến rất cao, cho phép người dùng xây dựng hệ thống quản lý thông tin cá nhân theo nhiều cách khác nhau. Tuy nhiên, sự linh hoạt này đôi khi khiến người dùng mới gặp khó khăn trong việc thiết lập hệ thống ban đầu. Theo lý thuyết Cognitive Load Theory, khi người dùng phải dành quá nhiều thời gian để thiết lập hệ thống trước khi sử dụng, điều này có thể tạo ra một dạng gánh nặng nhận thức (cognitive load) không cần thiết [7]. Từ những phân tích trên, có thể thấy rằng các công cụ hiện tại chưa thực sự tối ưu cho việc quản lý học tập của sinh viên. Điều này mở ra cơ hội phát triển một hệ thống

mới có khả năng tích hợp các tính năng cần thiết trong cùng một nền tảng.

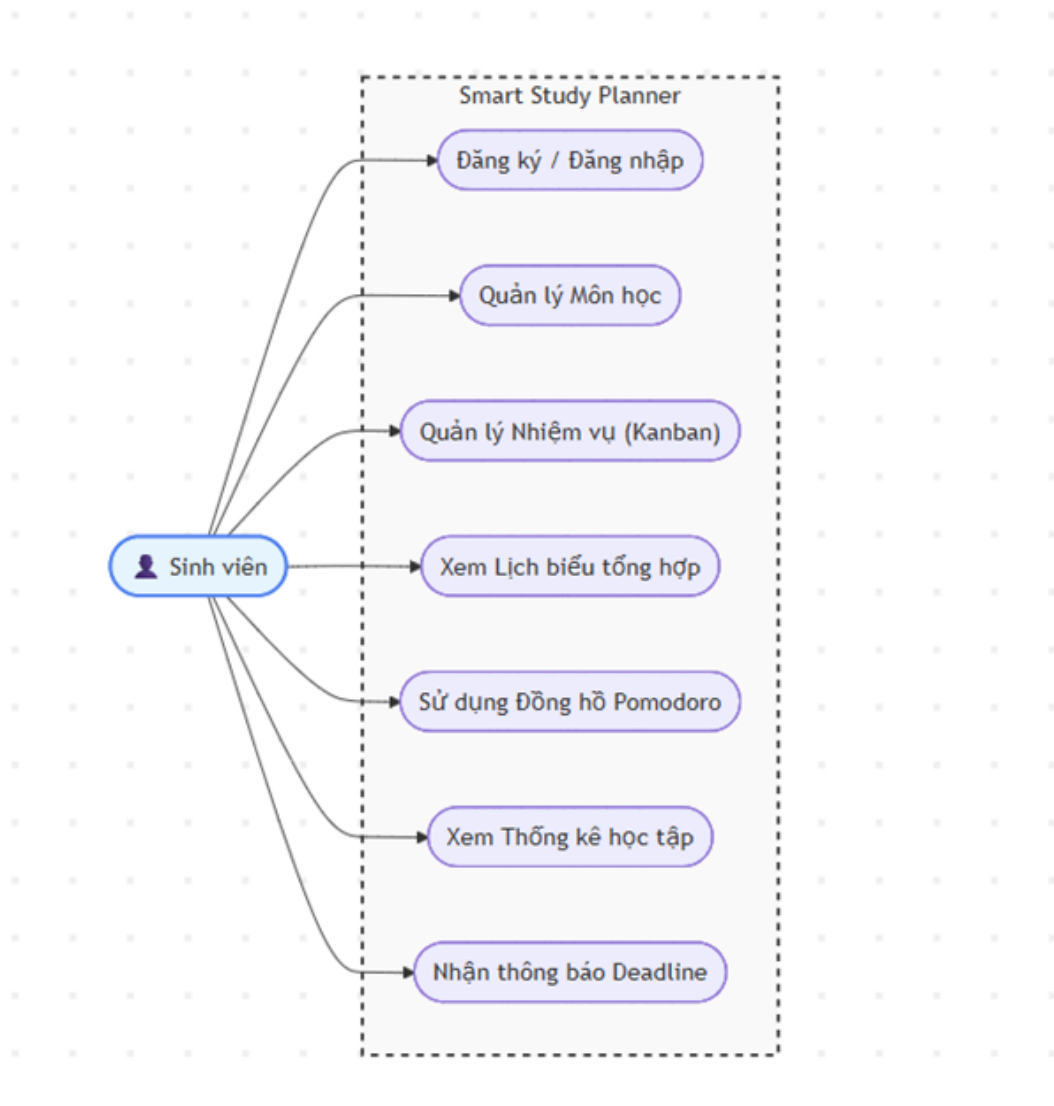
Từ những phân tích trên, có thể thấy rằng các công cụ hiện tại chưa thực sự tối ưu cho việc quản lý học tập của sinh viên. Điều này mở ra cơ hội phát triển một hệ thống mới có khả năng tích hợp các tính năng cần thiết trong cùng một nền tảng.

Tính năng	Google Calendar	Trello	Notion	Smart Study Planner
Lịch tổng hợp	Có	Không	Có	Có
Bảng Kan-ban	Không	Có	Có	Có
Tự động ưu tiên	Không	Không	Không	Có
Tích hợp Po-modoro	Không	Không	Không	Có
Dễ thiết lập	Có	Có	Không	Có

Bảng 2.1: Bảng phân tích và so sánh tính năng giữa các nền tảng quản lý thời gian hiện nay.

2.3 Phân tích và đặc tả yêu cầu hệ thống

Dựa trên kết quả khảo sát và phân tích hệ thống tham chiếu, hệ thống Smart Study Planner được thiết kế với bốn nhóm chức năng chính nhằm đáp ứng nhu cầu quản lý học tập của sinh viên.



Hình 2.1: Use case mô tả các tương tác của sinh viên với hệ thống

Nhóm chức năng Quản lý Dữ liệu Học tập (Core Modules): Đây là nhóm chức năng nền tảng của hệ thống, cho phép người dùng quản lý thông tin cá

nhân và dữ liệu học tập. Hệ thống sử dụng cơ chế xác thực dựa trên JSON Web Token (JWT) nhằm đảm bảo tính bảo mật trong quá trình đăng nhập. Ở cấp độ học vụ, người dùng có thể tạo và quản lý các môn học, đồng thời sử dụng cơ chế mã hóa màu sắc (color coding) để phân biệt các môn trên lịch học. Hệ thống cho phép thiết lập lịch học lặp lại và quản lý thông tin về giảng viên, phòng học. Đối với quản lý nhiệm vụ, mỗi bài tập được gắn các thông tin như deadline, mức độ ưu tiên và trạng thái tiến độ. Các nhiệm vụ này có thể được cập nhật trạng thái thông qua bảng Kanban để giúp người dùng theo dõi quá trình hoàn thành công việc.

Nhóm chức năng Bảng điều khiển và Hỗ trợ Lập kế hoạch: Hệ thống cung cấp Dashboard tổng quan nhằm hiển thị nhanh các thông tin học tập quan trọng như môn học sắp diễn ra, deadline gần nhất hoặc các nhiệm vụ cần hoàn thành. Để tăng động lực học tập, hệ thống áp dụng cơ chế Gamification thông qua tính năng theo dõi chuỗi ngày học tập liên tiếp (Study Streak). Chế độ Calendar View giúp người dùng quan sát toàn bộ lịch học và deadline dưới dạng các khối màu trực quan. Một trong những điểm nổi bật của hệ thống là module Smart Planner, cho phép tự động đề xuất kế hoạch học tập dựa trên các deadline và lịch thi đã nhập. Thuật toán ưu tiên nhiệm vụ sẽ phân tích thời gian còn lại trước mỗi deadline và đề xuất các khoảng thời gian học tập phù hợp.

Nhóm Công cụ Hỗ trợ Tập trung (Study Tools): Để giúp sinh viên duy trì sự tập trung khi học tập, hệ thống tích hợp trực tiếp bộ đếm thời gian Pomodoro vào giao diện ứng dụng. Trong quá trình học, người dùng có thể sử dụng danh sách kiểm tra (checklist) để chia nhỏ các nhiệm vụ lớn thành các bước nhỏ dễ thực hiện. Khi chế độ Pomodoro được kích hoạt, hệ thống có thể tạm dừng các thông báo không cần thiết nhằm giúp người dùng đạt trạng thái tập trung sâu (deep work).

Nhóm chức năng PWA và Thống kê nâng cao: Hệ thống được phát triển theo mô hình Progressive Web App (PWA) nhằm mang lại trải nghiệm tương tự ứng dụng di động mà không cần cài đặt từ cửa hàng ứng dụng. Theo nghiên cứu của Cherukuri (2024), các ứng dụng PWA có thể cải thiện đáng kể tỷ lệ giữ chân người dùng nhờ khả năng cài đặt trực tiếp từ trình duyệt [8]. Nhờ sử dụng Service Workers, hệ thống hỗ trợ chế độ Offline Mode, cho phép người dùng truy cập lịch học và nhiệm vụ ngay cả khi không có kết nối Internet. Khi kết nối được khôi phục, dữ liệu sẽ tự động đồng bộ với máy chủ. Ngoài ra, hệ thống còn cung cấp các biểu đồ thống kê học tập, giúp sinh viên theo dõi tiến độ hoàn thành nhiệm vụ và thời gian học trung bình theo từng tuần hoặc từng môn học.

Chương 3

THIẾT KẾ KIẾN TRÚC HỆ THỐNG VÀ THUẬT TOÁN

Trong quá trình xây dựng hệ thống Smart Study Planner, nhóm tập trung thiết kế một kiến trúc hệ thống đơn giản nhưng có khả năng mở rộng trong tương lai. Mục tiêu của kiến trúc này là hỗ trợ sinh viên quản lý nhiệm vụ học tập, theo dõi deadline và sắp xếp thời gian học tập một cách hiệu quả.

Nhìn chung, hệ thống lập kế hoạch học tập có thể được xem như một bài toán task scheduling. Trong bài toán này, các nhiệm vụ cần được sắp xếp theo mức độ ưu tiên và thời gian hoàn thành để tối ưu hóa hiệu quả sử dụng tài nguyên thời gian của người học. Theo các nghiên cứu gần đây trong lĩnh vực hệ thống hỗ trợ học tập và quản lý công việc cá nhân, việc áp dụng các heuristic scheduling algorithms có thể cải thiện đáng kể khả năng lập kế hoạch và ra quyết định của người dùng [16].

Ngoài ra, các hệ thống quản lý công việc hiện đại thường kết hợp cơ chế priority-based task management để hỗ trợ người dùng xác định công việc quan trọng nhất cần thực hiện trước [17]. Dựa trên hướng tiếp cận này, hệ thống Smart Study Planner được thiết kế với một thuật toán đánh giá mức độ ưu tiên tự động nhằm hỗ trợ sinh viên tổ chức quá trình học tập một cách khoa học và hiệu quả.

3.1 Kiến trúc tổng thể của hệ thống

Hệ thống được nhóm thiết kế theo mô hình Client - Server (Máy khách - Máy chủ) để tách biệt rõ ràng giữa phần xử lý dữ liệu và phần hiển thị giao diện.

Về phía Backend (Máy chủ), nhóm sử dụng môi trường Node.js kết hợp với framework Express.js. Việc lựa chọn công nghệ này giúp xây dựng một hệ thống RESTful API linh hoạt, tiếp nhận các yêu cầu từ phía người dùng (như thêm môn học, tạo công việc, lưu thời gian học), sau đó xử lý logic và trả về kết quả.

Để quản lý dữ liệu hiệu quả, nhóm tích hợp ORM Prisma. Công cụ này giúp việc tương tác với cơ sở dữ liệu trở nên trực quan và an toàn hơn so với việc viết các câu lệnh SQL truyền thống. Cơ sở dữ liệu của hệ thống được thiết kế xoay quanh bốn thực thể chính có mối quan hệ chặt chẽ với nhau. Cốt lõi là bảng Người dùng (User), nơi lưu trữ thông tin tài khoản. Mỗi người dùng sẽ tự quản lý danh sách Môn học (Subject) của riêng mình (bao gồm tên môn, màu sắc nhận diện, thời lượng học). Nằm bên trong mỗi môn học là các Công việc (Task) cần giải quyết, được theo dõi chi tiết từ mô tả, hạn chót (deadline) cho đến trạng thái (chưa làm, đang làm, đã xong). Cuối cùng, bảng Phiên học (Study Session) sẽ làm nhiệm vụ ghi nhận lại thời lượng thực tế mà người dùng đã bỏ ra cho từng công việc sau khi hoàn thành các chu kỳ học tập (ví dụ như học theo phương pháp Pomodoro).

Nhận thức được tầm quan trọng của việc bảo vệ dữ liệu cá nhân, nhóm đã xây dựng cơ chế bảo mật cho hệ thống ngay từ những bước đầu. Mật khẩu của người dùng khi đăng ký không được lưu trực tiếp mà sẽ trải qua quá trình băm (hashing) bằng thư viện bcrypt. Bên cạnh đó, hệ thống áp dụng cơ chế xác thực bằng JSON Web Token (JWT). Mỗi khi đăng nhập thành

công, người dùng được cấp một đoạn mã Token. Đoạn mã này đóng vai trò như một "chìa khóa" đi kèm trong mọi yêu cầu gửi lên máy chủ. Một phần mềm trung gian (Middleware verifyToken) sẽ đứng ra kiểm tra chìa khóa này. Nhờ vậy, hệ thống đảm bảo được tính riêng tư tuyệt đối: sinh viên nào chỉ được phép xem, chỉnh sửa và xóa dữ liệu môn học, công việc của chính sinh viên đó. Mô hình Client–Server được sử dụng rộng rãi trong các hệ thống web hiện đại nhờ khả năng tách biệt giữa giao diện người dùng và tầng xử lý dữ liệu. Kiến trúc này giúp hệ thống dễ dàng mở rộng, bảo trì và triển khai trên nhiều nền tảng khác nhau [18]. Trong các hệ thống quản lý học tập và productivity tools hiện nay, backend thường được triển khai dưới dạng RESTful API để cho phép nhiều ứng dụng khách (web hoặc mobile) truy cập cùng một nguồn dữ liệu chung [19].

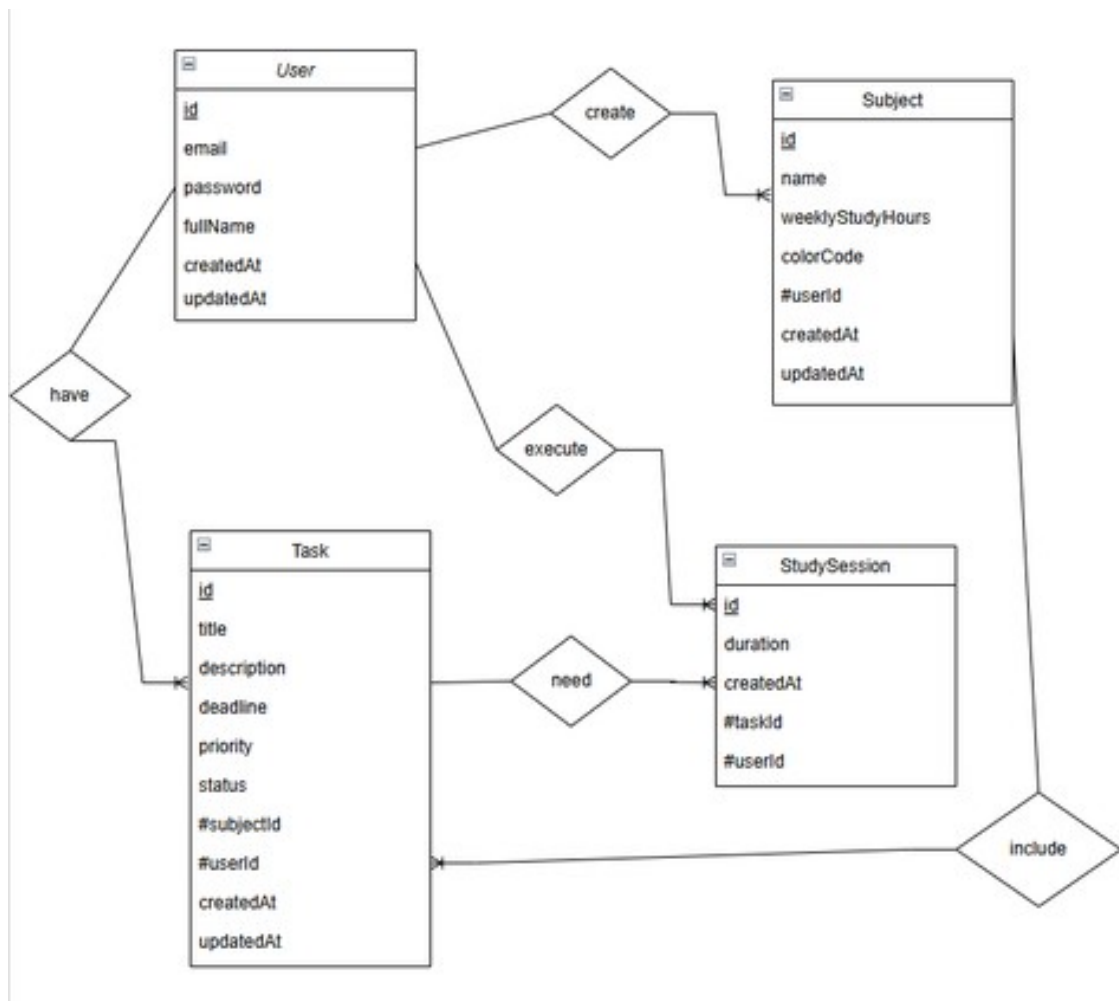
3.2 Thiết kế cấu trúc cơ sở dữ liệu

Một trong những mục tiêu quan trọng của ứng dụng là giúp sinh viên giảm bớt gánh nặng trong việc tự sắp xếp kế hoạch. Do đó, thay vì yêu cầu người dùng phải tự đánh giá và dán nhãn mức độ quan trọng cho từng bài tập, nhóm đã thiết kế một thuật toán tự động tính toán độ ưu tiên (Auto-Priority Algorithm). Cụ thể, khi người dùng tạo mới hoặc cập nhật một công việc, họ chỉ cần nhập thời gian nộp bài (Deadline). Tại Backend, hàm calculateAutoPriority sẽ tự động tính toán khoảng cách từ thời điểm hiện tại cho đến lúc hết hạn, từ đó phân loại và gán mức độ ưu tiên phù hợp (ví dụ: việc cần làm gấp, việc có thể làm sau). Điều này giúp danh sách công việc luôn được tự động sắp xếp một cách khoa học.

Bên cạnh thuật toán ưu tiên, hệ thống còn tập trung mạnh vào các logic thống kê dữ liệu để cung cấp cái nhìn tổng quan cho người dùng. Khi truy

xuất danh sách môn học hoặc xem bảng điều khiển (Dashboard), Backend không chỉ trả về dữ liệu thô mà còn thực hiện các phép tính ngầm. Hệ thống sẽ quét toàn bộ công việc trong tuần (từ thứ Hai đến Chủ nhật), đếm số lượng công việc đã hoàn thành so với tổng số lượng được giao, từ đó tính ra phần trăm tiến độ học tập của tuần đó. Tương tự, dữ liệu về thời gian học cũng được cộng dồn theo từng ngày và quy đổi từ phút sang giờ để chuẩn bị sẵn sàng cho việc vẽ biểu đồ trực quan.

Trong thời gian tới, để hệ thống trở nên chủ động và thông minh hơn, nhóm dự kiến sẽ nghiên cứu tích hợp thêm cơ chế chạy ngầm (Cron Job). Tiến trình này dự kiến sẽ hoạt động tự động theo định kỳ (ví dụ như quét hệ thống vào lúc nửa đêm) để rà soát các công việc đã quá hạn chót mà chưa hoàn thành, từ đó tự động cập nhật trạng thái hoặc chuẩn bị dữ liệu cho tính năng gửi thông báo nhắc nhở người dùng.



Hình 3.1: Sơ đồ thực thể - Quan hệ (ERD) của cơ sở dữ liệu

Cấu trúc chi tiết của các bảng dữ liệu được đặc tả như sau:

Chi tiết thực thể User (Người dùng) Đóng vai trò hạt nhân của hệ thống, lưu trữ thông tin định danh cá nhân và xác thực bảo mật:

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Ràng buộc	Mô tả chi tiết
id	UUID	PK	Unique, Not Null	Mã định danh người dùng
email	String		Unique, Not Null	Email đăng nhập
password	String		Not Null	Mật khẩu đã hash bằng Bcrypt
fullName	String		Not Null	Họ và tên người dùng
createdAt	DateTime		Default(now())	Thời điểm tạo tài khoản
updatedAt	DateTime		AutoUpdate	Thời điểm cập nhật gần nhất

Bảng 3.1: Chi tiết thực thể User (Người dùng)

Chi tiết thực thể Subject (Môn học) Hoạt động như một nhóm phân loại (Category) cho toàn bộ các hoạt động học vụ. Có mối quan hệ một-nhiều (1-N) với bảng User.

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Ràng buộc	Mô tả chi tiết
id	UUID	PK	Unique, Not Null	Mã định danh môn học
name	String		Not Null	Tên môn học
weeklyStudyHours	int		Default(4)	Số giờ học dự kiến mỗi tuần
colorCode	String		Nullable	Mã màu hiển thị trên giao diện
userId	UUID	FK	Tham chiếu user	Người sở hữu môn học
createdAt	DateTime		Default(now())	Thời điểm tạo
updatedAt	DateTime		Auto update	Thời điểm cập nhật

Bảng 3.2: Chi tiết thực thể Subject (Môn học)

Chi tiết thực thể Task (Nhiệm vụ học tập) Lưu trữ dữ liệu động phức tạp nhất, dùng để quản lý chi tiết bài tập. Có mối quan hệ khóa ngoại (1-N) với cả bảng Subject và bảng User.

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Ràng buộc	Mô tả chi tiết
id	UUID	PK	Unique, Not Null	Mã định danh duy nhất của nhiệm vụ
title	String		Not Null	Tên bài tập, đề án
description	text		Nullable	Mô tả chi tiết yêu cầu bài tập
deadline	Datetime		Not Null	Thời hạn nộp bài
status	String		Default ('TODO')	Trạng thái thẻ Kanban(TODO, IN PROGRESS, DONE)
priority	int		Default(2)	Độ ưu tiên: 1(High), 2(Medium), 3(Low) được Cronjob cập nhật
subjectId	Nullable	FK	Tham chiếu Subject	Định danh môn học chứa bài tập này
createdAt	DateTime		Default(now())	Thời điểm tạo
userId	UUID	FK	Tham chiếu user	Định danh sinh viên sở hữu bài tập này

Bảng 3.3: Chi tiết thực thể Task (Nhiệm vụ học tập)

Chi tiết thực thể StudySession (Phiên học): Thực thể StudySession được

sử dụng để ghi nhận thời gian học thực tế của sinh viên. Mỗi phiên học thường tương ứng với một chu kỳ Pomodoro hoặc một khoảng thời gian tập trung. Dữ liệu này được lưu trữ dưới dạng chuỗi thời gian (time-series data) để phục vụ cho việc phân tích thống kê và hiển thị biểu đồ tiến độ học tập trong tuần.

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Ràng buộc	Mô tả chi tiết
id	UUID	PK	Unique, Not Null	Mã định danh của phiên học
duration	int		Not Null	Thời lượng học (tính bằng phút)
createdAt	DateTime		Not Null	Ngày diễn ra phiên học
userId	UUID	FK	Tham chiếu user	Sinh viên thực hiện phiên học
taskId	UUID	FK	Tham chiếu task	Nhiệm vụ được học trong phiên

Bảng 3.4: Chi tiết thực thể Student Session (Phiên học)

3.3 Thiết kế hệ thống API

Backend cung cấp kiến trúc RESTful API, chia thành các nhóm chức năng độc lập để Frontend dễ dàng tích hợp:

- Nhóm API Xác thực (Auth): Xử lý đăng ký, đăng nhập và đổi mật khẩu.
- Nhóm API Môn học (Subject): Hỗ trợ thêm, sửa và thống kê môn học.

Đặc biệt, API lấy danh sách môn học đã được tích hợp logic tự động tính toán phần trăm tiến độ dựa trên số lượng Task đã hoàn thành (công thức: $(\text{Số task hoàn thành} / \text{Tổng số task}) * 100$).

Nhóm API Công việc (Task): Xử lý vòng đời của một bài tập từ lúc tạo mới đến lúc chuyển trạng thái hoàn thành. API này cũng cung cấp dữ liệu tiến độ tuần (Weekly Progress) dựa trên khối lượng công việc và thời gian ước tính.

Nhóm API Thống kê (Study Session): Ghi nhận thời gian học (kết hợp với đồng hồ Pomodoro) và cung cấp dữ liệu định dạng chuẩn để Frontend vẽ biểu đồ thời gian học trong 7 ngày của tuần.

Cấu trúc định tuyến của các API cốt lõi được liệt kê trong bảng dưới đây:

Bảng 3.5: Danh sách các RESTful API cốt lõi của hệ thống

Nhóm chức năng	Phương thức	Endpoint (Đường dẫn)	Yêu cầu JWT	Mô tả chức năng
Auth	POST	/api/v1/auth/register	Không	Đăng ký tài khoản sinh viên mới, hệ thống băm mật khẩu bằng Bcrypt trước khi lưu.
Auth	POST	/api/v1/auth/login	Không	Đăng nhập hệ thống, server xác thực mật khẩu và cấp JWT Token.

Bảng 3.5 – Danh sách các RESTful API cốt lõi của hệ thống

Nhóm chức năng	Phương thức	Endpoint (Đường dẫn)	Yêu cầu JWT	Mô tả chức năng
Auth	PUT	/api/v1/auth/forgot-password	Có	Người dùng đã đăng nhập có thể thay đổi mật khẩu của mình.
Task	POST	/api/v1/tasks/createTask	Có	Tạo nhiệm vụ học tập mới và tự động tính độ ưu tiên dựa trên deadline.
Task	GET	/api/v1/tasks/getTask	Có	Lấy danh sách tất cả các nhiệm vụ của người dùng để hiển thị trên bảng Kanban và lịch học.
Task	PATCH	/api/v1/tasks/updateStatus/:id	Có	Cập nhật trạng thái nhiệm vụ (TODO - IN PROGRESS - DONE).
Task	PUT	/api/v1/tasks/updateTask/:id	Có	Cập nhật nội dung nhiệm vụ như tiêu đề, deadline hoặc mô tả.

Bảng 3.5 – Danh sách các RESTful API cốt lõi của hệ thống

Nhóm chức năng	Phương thức	Endpoint (Đường dẫn)	Yêu cầu JWT	Mô tả chức năng
Task	DELETE	/api/v1/tasks/weekly-progress	Có	Trả về dữ liệu tiến độ học tập trong tuần để hiển thị biểu đồ thống kê.
StudySession	POST	/api/v1/studySession/saveTime	Có	Lưu thời gian học khi hoàn thành một phiên Pomodoro.
StudySession	GET	/api/v1/studySession/weekly-chart	Có	Lấy dữ liệu thời gian học trong 7 ngày để hiển thị biểu đồ Dashboard.
User	DELETE	/api/v1/user/delete	Có	Xóa tài khoản của người dùng cùng toàn bộ dữ liệu liên quan.

3.4 Thuật toán phân loại độ ưu tiên dựa trên ranh giới thời gian

Trong lĩnh vực hệ thống thông tin và quản trị tác vụ, việc xác định thứ tự thực hiện công việc thường được định hướng bởi các ràng buộc về thời gian (time constraints). Một trong những tiêu chuẩn nền tảng phổ biến nhất là

thuật toán Earliest Deadline First (EDF) . Theo nguyên lý này, mức độ cấp bách (urgency) của một nhiệm vụ tỷ lệ nghịch với khoảng cách thời gian từ thời điểm hiện tại cho đến hạn chót (deadline); nhiệm vụ có hạn chót càng gần sẽ tự động được gán độ ưu tiên càng cao nhằm tối ưu hóa tỷ lệ hoàn thành đúng hạn (deadline adherence rate).

Tuy nhiên, thay vì chỉ sắp xếp mǎng công việc một cách tuyến tính và liên tục theo thời gian, các hệ thống quản lý hiện đại thường lượng hóa khoảng thời gian này thành các phân khúc rời rạc (discrete tiers) để giảm tải độ phức tạp tính toán và tăng tính trực quan cho người dùng. Bằng cách thiết lập các "ranh giới thời gian" (time boundaries) tiêu chuẩn—tương tự như cách tiếp cận trực "Khẩn cấp" trong hệ thống Ma trận Eisenhower—hệ thống có thể tự động ánh xạ lượng thời gian còn lại thành các giá trị ưu tiên phân cấp rõ ràng (ví dụ: Cao, Trung bình, Thấp).

Dựa trên cơ sở lý thuyết này, cơ chế đánh giá độ khẩn cấp của công việc được mô hình hóa toán học thông qua một hàm phân mảnh (piecewise function). [20]

Công thức phân loại độ ưu tiên theo ranh giới thời gian (Time-based Priority Function)

Hệ thống xác định độ ưu tiên của một công việc (P) dựa trên một hàm phân mảnh với biến số là khoảng cách thời gian từ thời điểm hiện tại đến hạn chót của công việc (d). Công thức được biểu diễn như sau:

$$P(\Delta d) = \begin{cases} 1. \text{ Nếu } \Delta d \leq 2 \\ 2. \text{ Nếu } 2 < \Delta d \leq 7 \\ 3. \text{ Nếu } \Delta d > 7 \text{ hoặc không xác định } \Delta d \end{cases} \quad (3.1)$$

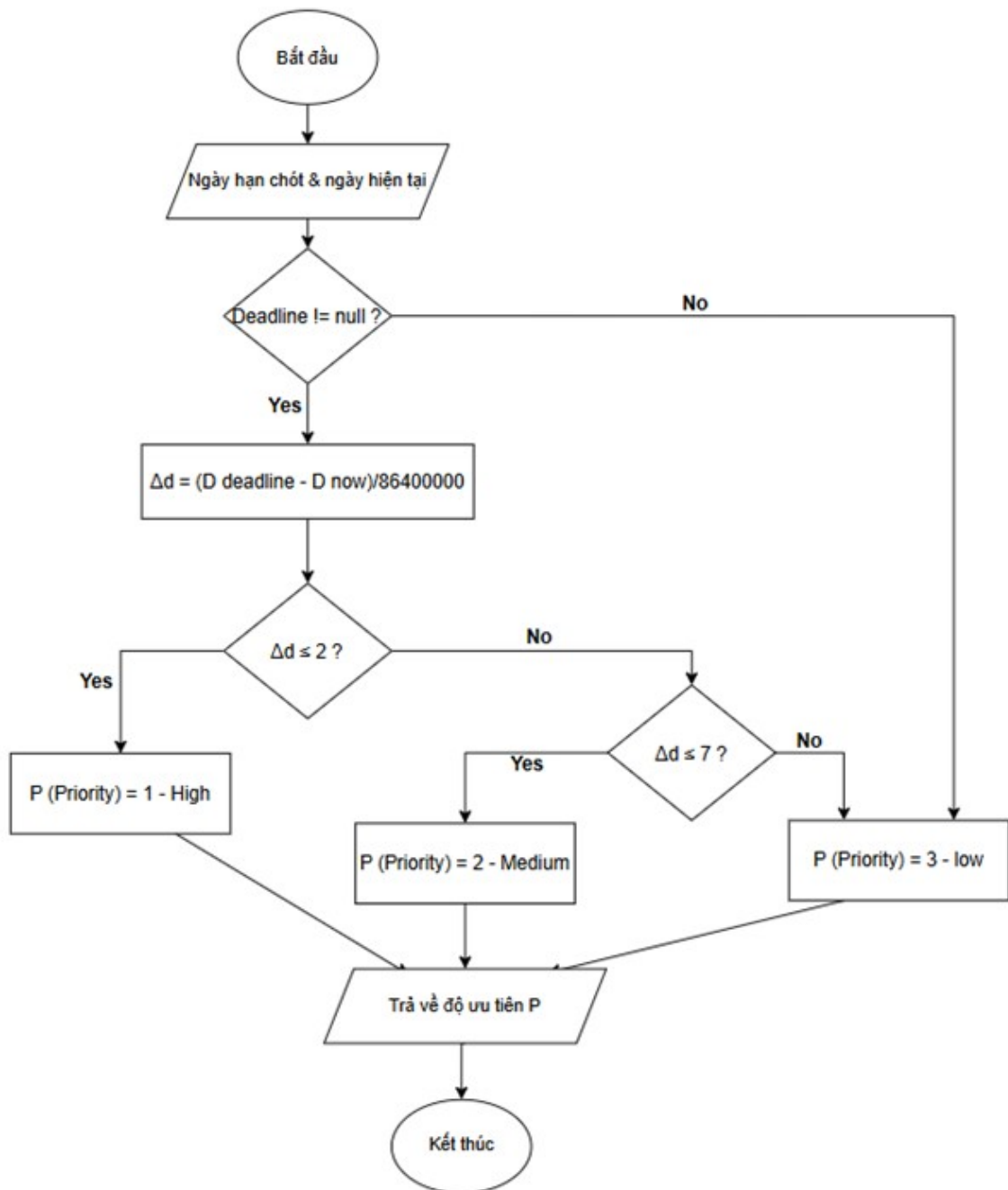
Trong đó:

- P (Priority): Là mức độ ưu tiên của công việc được chuẩn hóa thành

các giá trị nguyên $\{1, 2, 3\}$. Giá trị càng nhỏ, độ ưu tiên càng cao (1 = High, 2 = Medium, 3 = Low).

- Δd (Delta days): Là số ngày còn lại cho đến hạn chót, được tính bằng hiệu số giữa ngày của hạn chót (D_{deadline}) và ngày hiện tại (D_{now}), đã được làm tròn về 0 giờ 0 phút 0 giây để loại bỏ sai số về múi giờ.

$$\Delta d = \frac{D_{\text{deadline}} - D_{\text{now}}}{86400000} \quad (3.2)$$



Hình 3.2: Lưu đồ thuật toán tính toán mức độ ưu tiên dựa trên ranh giới thời gian

Cơ chế logic toán học này giúp hệ thống tự động điều chỉnh mức độ ưu tiên của các công việc khi quỹ thời gian cạn dần, từ đó liên tục điều hướng sự chú ý của người dùng vào các "điểm nóng" quan trọng nhất.

3.5 Thuật toán đánh giá tiến độ dựa trên thời lượng (Time-Weighted Progress Algorithm)

Bên cạnh việc phân loại ưu tiên, hệ thống còn tích hợp Kỹ thuật Quản lý Thời gian Pomodoro vào kiến trúc Cơ sở dữ liệu Chuỗi thời gian (Time-series Data) nhằm đo lường chính xác cường độ học tập. Thay vì chỉ ghi nhận trạng thái công việc (Hoàn thành/Chưa hoàn thành), hệ thống phân rã thời gian học thành các 'Phiên học' (Study Sessions). Quá trình này cung cấp một tập dữ liệu định lượng (Quantitative Data) liên tục, cho phép thuật toán tổng hợp (Aggregation) tự động quy đổi và ánh xạ thời gian thực tế thành biểu đồ trực quan. Điều này giúp sinh viên phản chiếu (Reflect) và điều chỉnh hành vi phân bổ thời gian của mình trong suốt cả tuần.

Trong quá trình đánh giá hiệu suất học tập (Performance Evaluation), nhóm nhận thấy việc tính toán tiến độ dựa trên số lượng công việc (Task-based Calculation) truyền thống thường dẫn đến hiệu ứng đánh lừa nỗ lực (Effort Illusion). Sinh viên có xu hướng hoàn thành các công việc nhỏ lẻ để làm đầy thanh tiến độ mà bỏ qua các nhiệm vụ trọng tâm đòi hỏi nhiều thời gian. Do đó, hệ thống của nhóm đề xuất và song hành áp dụng thêm Thuật toán đánh giá Tiến độ dựa trên Thời lượng (Time-weighted Progress Algorithm). Phương pháp này gán trọng số (thời gian dự kiến/mục tiêu giờ học) cho từng công việc, giúp phản ánh chính xác 100% lượng công sức thực tế mà sinh viên đã bỏ ra.

Thuật toán này được hệ thống xử lý và trả về dưới dạng JSON thông qua endpoint GET `/api/v1/studySession/weekly-chart`, cung cấp nguồn dữ liệu phong phú để thư viện Recharts kết xuất thành các biểu đồ thống kê sinh

động trên giao diện Dashboard.

3.6 Cơ chế cập nhật tự động bằng tiến trình chạy ngầm (Cron Job)

Trong các hệ thống phần mềm hiện đại, nhiều tác vụ cần được thực hiện tự động theo chu kỳ thời gian mà không yêu cầu sự can thiệp trực tiếp của người dùng. Các tiến trình dạng này thường được triển khai dưới dạng background jobs hoặc scheduled tasks. Theo các nghiên cứu về kiến trúc hệ thống web, việc sử dụng cơ chế lập lịch tác vụ định kỳ giúp hệ thống duy trì tính nhất quán của dữ liệu và giảm tải cho các thao tác phía người dùng [21]. Vì lý do đó, hệ thống Smart Study Planner triển khai một tiến trình chạy nền (Cron Job) trên máy chủ để tự động cập nhật mức độ ưu tiên của các nhiệm vụ theo thời gian.

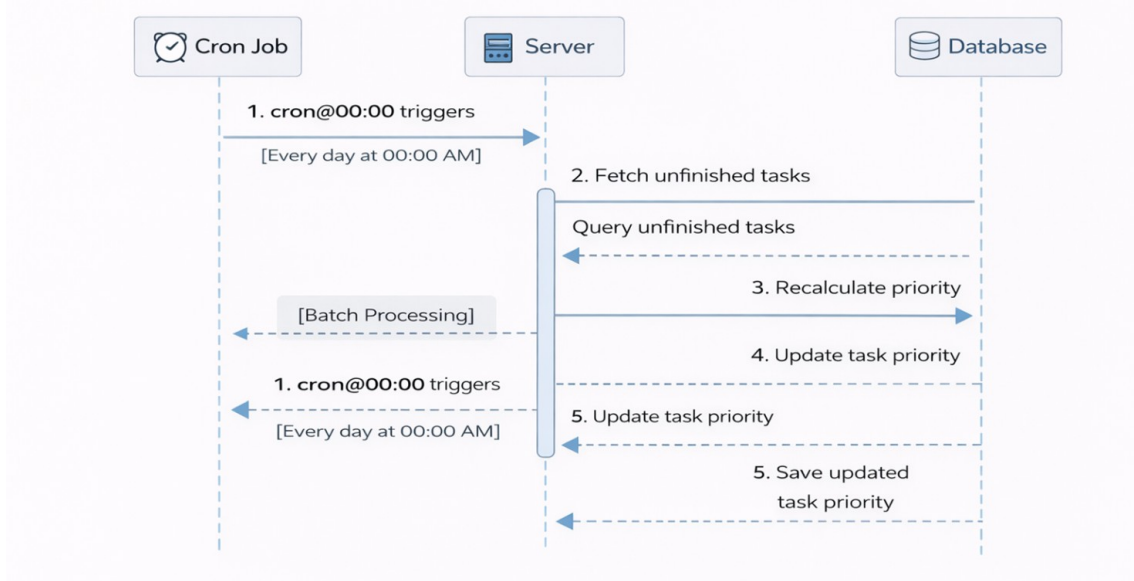
Trong quá trình thiết kế hệ thống, nhóm nhận thấy một vấn đề thực tế: khoảng cách thời gian từ hiện tại cho đến hạn chót (deadline) của các công việc sẽ tự động ngắn dần theo từng ngày. Nếu chỉ tính toán độ ưu tiên một lần duy nhất lúc người dùng mới tạo công việc, dữ liệu sẽ nhanh chóng bị lỗi thời. Để giải quyết triệt để vấn đề này mà không bắt người dùng phải tải lại trang hay thao tác thủ công, nhóm đã phát triển thêm một cơ chế chạy ngầm sử dụng thư viện node-cron. Cơ chế này hoạt động như một "người quản gia" thầm lặng của hệ thống với quy trình như sau:

- Thiết lập lịch trình: Tiến trình được cài đặt để tự động kích hoạt vào lúc 00:00 (nửa đêm) mỗi ngày. Thời điểm này được chọn vì đây là lúc hệ thống ít người truy cập nhất, giúp hạn chế ảnh hưởng đến trải nghiệm chung.
- Quét và lọc dữ liệu: Khi đến giờ, hệ thống sẽ tự động quét toàn bộ cơ sở dữ liệu để tìm ra những công việc vẫn đang ở trạng thái chưa hoàn thành

(khác "DONE") và có đặt ngày đến hạn (deadline).

- **Đánh giá và cập nhật:** Với danh sách công việc vừa tìm được, hệ thống sẽ sử dụng lại thuật toán `calculateAutoPriority` để tính toán xem mức độ ưu tiên của công việc đó ở thời điểm ngày hôm nay là bao nhiêu. Thay vì cập nhật mà quét toàn bộ dữ liệu, thuật toán sẽ so sánh mức độ ưu tiên mới với mức độ ưu tiên cũ. Chỉ những bài tập thực sự có sự thay đổi về cấp độ (ví dụ: chuyển từ mức Trung bình lên mức Cao do sắp đến hạn) thì mới được hệ thống ghi đè vào cơ sở dữ liệu.

Nhờ có tiến trình chạy ngầm này, mỗi buổi sáng khi sinh viên đăng nhập vào hệ thống, danh sách bài tập đã được làm mới một cách âm thầm. Những công việc sắp đến hạn sẽ tự động được đẩy lên nhóm ưu tiên cao nhất, giúp người dùng luôn nắm bắt được đúng tiến độ học tập.



Hình 3.3: Sơ đồ tuần tự (Sequence Diagram) mô tả luồng

Cơ chế này được thiết lập biểu thức định thời để tự động kích hoạt chính xác vào lúc 00:00 (nửa đêm) mỗi ngày. Khi tiến trình bắt đầu, hệ thống thực hiện luồng xử lý hàng loạt (batch processing): Trước tiên, nó truy vấn và

trích xuất danh sách toàn bộ các nhiệm vụ đang ở trạng thái chưa hoàn thành trong cơ sở dữ liệu. Sau đó, nó áp dụng thuật toán đã thiết kế ở mục 3.4 để tính toán lại mức độ ưu tiên mới cho từng nhiệm vụ dựa trên quỹ thời gian vừa bị trừ đi 1 ngày. Cuối cùng, hệ thống đối chiếu và thực hiện lệnh cập nhật (Update) giá trị priority mới vào cơ sở dữ liệu nếu có sự thay đổi cấp độ. Nhờ luồng xử lý nền (Background processing) diễn ra vào giờ thấp điểm này, dữ liệu luôn được làm mới sẵn sàng cho ngày học tập tiếp theo mà không làm ảnh hưởng đến hiệu năng truy cập chung của hệ thống.

3.7 Thiết kế giao diện người dùng (Frontend Architecture)

diện người dùng của hệ thống được xây dựng bằng thư viện React, kết hợp với cơ chế điều hướng của thư viện React Router để quản lý các trang chức năng trong ứng dụng. Cách tiếp cận này cho phép xây dựng một Single Page Application (SPA), trong đó việc chuyển đổi giữa các trang được thực hiện ở phía trình duyệt mà không cần tải lại toàn bộ trang web.

Trong hệ thống Smart Study Planner, cấu trúc định tuyến được thiết kế thông qua hàm `createBrowserRouter`. Router này định nghĩa các đường dẫn chính của ứng dụng, bao gồm các trang xác thực người dùng như đăng nhập, đăng ký và khôi phục mật khẩu, cũng như các trang chức năng chính của hệ thống.

Các trang chức năng chính được đặt bên trong một thành phần Layout chung, bao gồm:

- Dashboard: hiển thị tổng quan các nhiệm vụ và tiến độ học tập.
- Tasks: quản lý danh sách công việc học tập.
- Calendar: hiển thị lịch học và các deadline.

- Subjects: quản lý môn học.
- Pomodoro: công cụ hỗ trợ tập trung học tập.
- Analytics: thống kê hiệu quả học tập.
- Profile và Settings: quản lý thông tin cá nhân và cấu hình hệ thống.

Việc tổ chức router theo cấu trúc phân cấp giúp mã nguồn dễ bảo trì và mở rộng trong tương lai. Ngoài ra, mô hình SPA cũng giúp cải thiện trải nghiệm người dùng nhờ giảm thời gian tải trang và tăng tốc độ phản hồi của giao diện [22].

Chương 4

KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày quá trình triển khai hệ thống quản lý học tập thông minh Smart Study Planner được đề xuất trong đề tài. Đồng thời, chương cũng mô tả chi tiết môi trường thực nghiệm, phân tích sâu kết quả triển khai các phân hệ chức năng trên giao diện người dùng và tiến hành đánh giá hiệu quả thực tiễn của thuật toán Auto-Priority kết hợp cơ chế Cron Job thông qua các kịch bản kiểm thử (test cases) cụ thể.

4.1 Môi trường và công cụ triển khai hệ thống

Khác với các hệ thống học máy phụ thuộc vào tập dữ liệu tĩnh (static datasets), hệ thống phần mềm ứng dụng trong đề tài này được đánh giá dựa trên môi trường triển khai thực tế và các kịch bản tương tác động của người dùng. Hệ thống được triển khai và thử nghiệm trên môi trường phát triển cục bộ (localhost) với cấu hình phần cứng và phần mềm được chuẩn hóa nhằm mô phỏng điều kiện vận hành của một máy chủ web tiêu chuẩn.

Thành phần	Công nghệ / Thư viện sử dụng	Mục đích và chi tiết triển khai
Phần cứng	CPU Intel Core i5 / AMD Ryzen 5	RAM 8GB - 16GB, Ổ cứng SSD.
Frontend	ReactJS (v18) + Vite	Xây dựng kiến trúc Single Page Application (SPA) với tốc độ tải trang siêu tốc nhờ Virtual DOM.
UI Framework	Tailwind CSS + Radix UI	Thiết kế giao diện (Utility-first) và cung cấp các Headless Component đảm bảo khả năng truy cập.
Backend	Node.js + Express.js	Khởi tạo máy chủ xử lý I/O bất đồng bộ và cung cấp các RESTful API độc lập.
Cơ sở dữ liệu	PostgreSQL + Prisma ORM	Quản trị CSDL quan hệ, tối ưu hóa truy vấn và đảm bảo tính toàn vẹn dữ liệu qua khóa ngoại.
Bảo mật	JWT + Bcrypt	Mã hóa mật khẩu một chiều (One-way hashing) và xác thực phi trạng thái (Stateless Authentication).
Tiến trình	node-cron	Thiết lập tiến trình chạy ngầm xử lý hàng loạt (Batch Processing) tự động lúc 00:00 mỗi ngày.

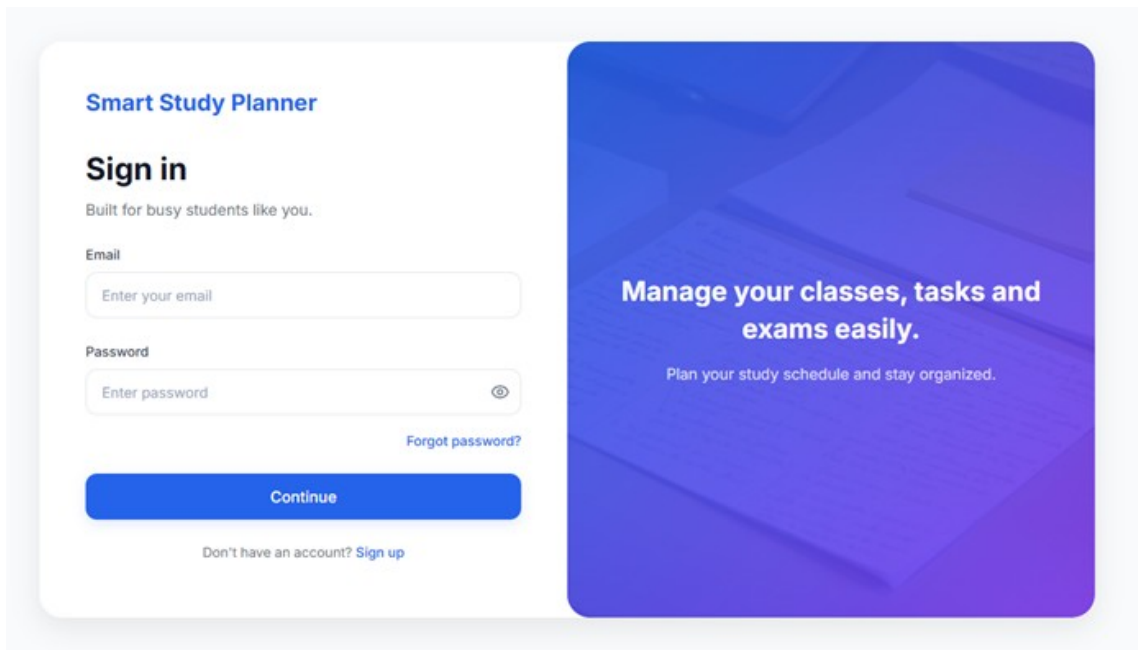
Bảng 4.1: Cấu hình môi trường thực nghiệm và phát triển hệ thống

Phương pháp thực nghiệm được tiến hành qua hai giai đoạn: (1) Kiểm thử chức năng (Functional Testing) để đảm bảo các luồng dữ liệu CRUD (Tạo, Đọc, Cập nhật, Xóa) hoạt động chính xác; và (2) Kiểm thử logic thuật toán (Algorithm Testing) để xác minh cơ chế cập nhật tự động mức độ ưu tiên của hệ thống theo thời gian thực.

4.2 Kết quả triển khai các phân hệ chức năng

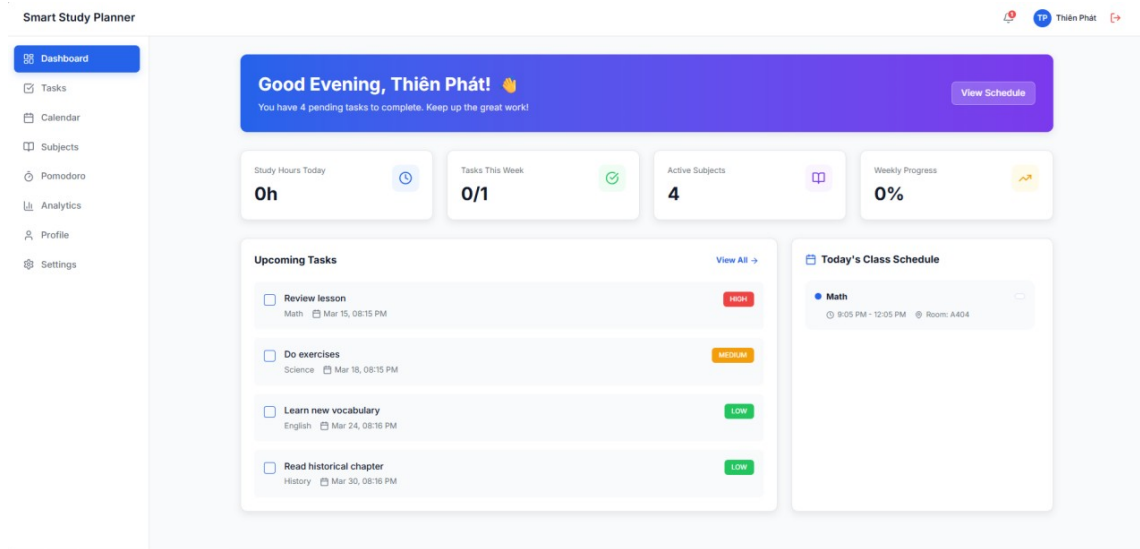
Sau quá trình lập trình và kiểm thử liên tục, hệ thống Smart Study Planner đã hiện thực hóa thành công các yêu cầu chức năng đặt ra ban đầu, mang đến một hệ sinh thái học tập khép kín, mượt mà và trực quan.

Phân hệ Quản lý Tài khoản và Bảo mật: Hệ thống đã triển khai hoàn thiện luồng đăng ký và đăng nhập. Khi người dùng khởi tạo tài khoản, thuật toán Bcrypt tự động băm mật khẩu thành các chuỗi ký tự không thể dịch ngược trước khi lưu vào PostgreSQL. Các biểu mẫu được quản lý bằng React Hook Form kết hợp với biểu thức chính quy (Regex) để kiểm duyệt tính hợp lệ của dữ liệu (ví dụ: độ dài mật khẩu, định dạng email) ngay tại thời điểm nhập liệu. Quá trình đăng nhập thành công sẽ trả về mã JWT có thời hạn 7 ngày, giúp sinh viên duy trì phiên làm việc bảo mật mà không bị gián đoạn.



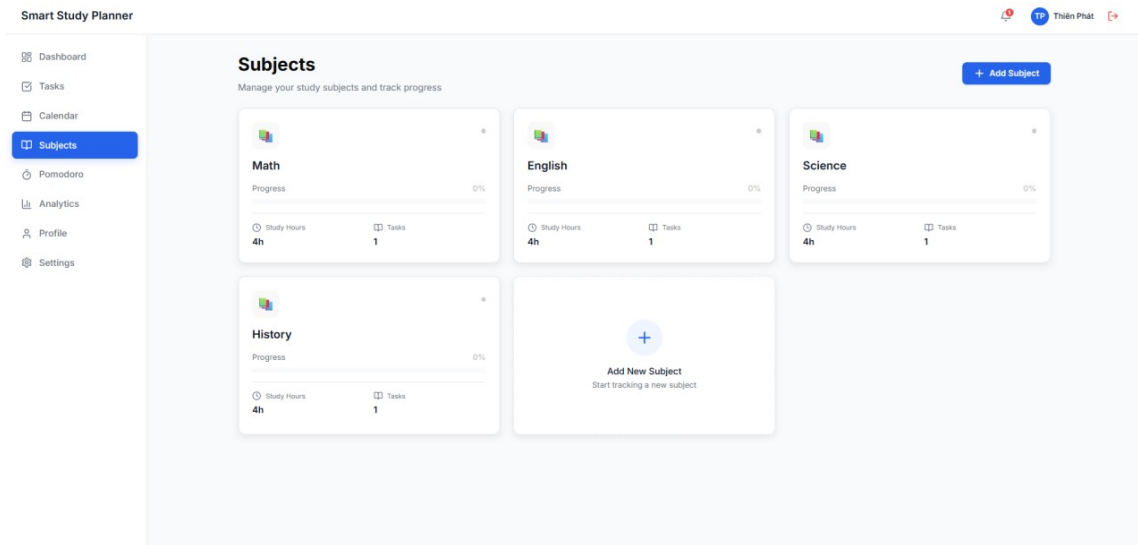
Hình 4.1: Giao diện đăng nhập và xác thực người dùng tích hợp kiểm duyệt lỗi trực tiếp

Phân hệ Quản lý Môn học và Lịch biểu tổng quan: Giao diện quản lý môn học cho phép sinh viên khởi tạo dữ liệu với các trường tùy biến như tên môn, số giờ học dự kiến mỗi tuần (`weeklyStudyHours`) và mã màu (`color-coding`). Việc sử dụng mã màu giúp người dùng dễ dàng phân loại thị giác và được đồng bộ trực tiếp lên giao diện Calendar và Dashboard. Giao diện Dashboard được thiết kế theo dạng lưới (`Grid layout`) hiện đại, tổng hợp các số liệu thống kê nhanh và hiển thị một thanh tiến trình (`Progress bar`) tự động tính toán tỷ lệ phần trăm bài tập đã hoàn thành của từng môn.



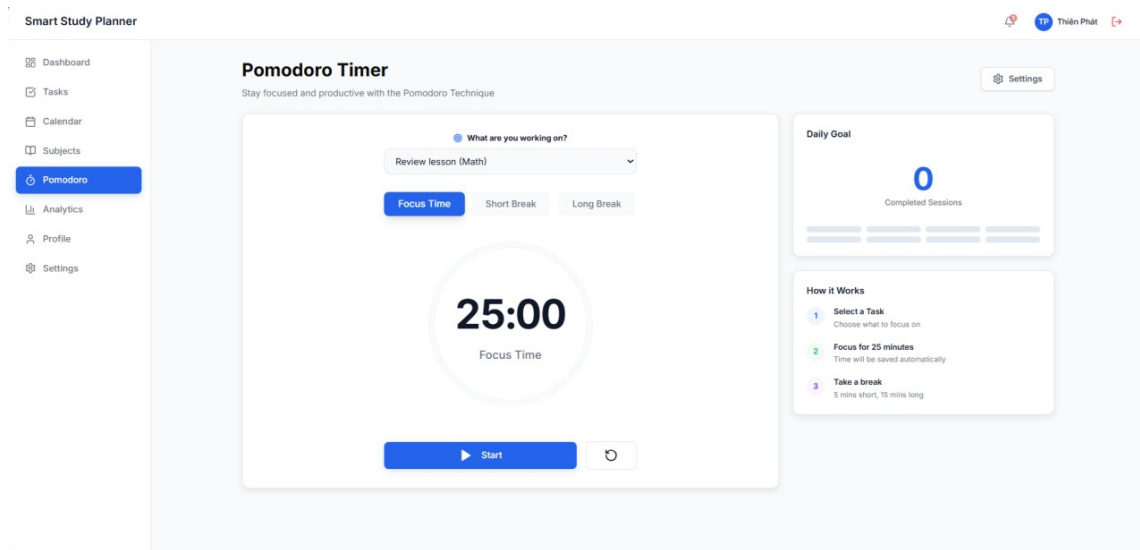
Hình 4.2: Giao diện bảng điều khiển (DashBoard) tổng quan tiến độ học tập

Phân hệ Quản lý Nhiệm vụ và Bảng Kanban: Đây là phân hệ cốt lõi giải quyết bài toán tổ chức công việc. Dữ liệu bài tập được kết xuất dưới dạng các thẻ công việc trực quan trên bảng Kanban với ba cột trạng thái chuẩn mực: Cần thực hiện (TODO), Đang thực hiện (IN PROGRESS) và Đã hoàn thành (DONE). Hệ thống áp dụng kỹ thuật Optimistic UI Update (Cập nhật giao diện lạc quan) kết hợp với thư viện React DnD, cho phép người dùng kéo thả thẻ mượt mà; giao diện sẽ thay đổi ngay lập tức song song với việc API âm thầm cập nhật vào cơ sở dữ liệu. Theo nghiên cứu của Mahnic (2019) về việc ứng dụng Agile trong giáo dục, cấu trúc Kanban này giúp trực quan hóa luồng công việc (workflow) và giới hạn khối lượng công việc đang thực hiện (WIP limits), từ đó giúp sinh viên khắc phục triệt để sự rối rắm của các danh sách dạng tĩnh truyền thống [23].



Hình 4.3: Quản lý vòng đời bài tập bằng mô hình bảng Kanban

Phân hệ Hỗ trợ Tập trung (Pomodoro) và Thống kê (Analytics): Hệ thống đã nhúng thành công bộ đếm thời gian Pomodoro đếm ngược 25 phút ngay bên trong ứng dụng. Việc tích hợp này loại bỏ sự phân tán ngữ cảnh do phải chuyển đổi sang các ứng dụng của bên thứ ba. Sau mỗi phiên tập trung, dữ liệu về thời lượng được hệ thống gửi lên máy chủ và xử lý thành chuỗi thời gian (time-series) trên trang Analytics thông qua thư viện Recharts. Hơn thế nữa, theo nghiên cứu mới nhất của Tổ chức Y tế Công cộng Hoa Kỳ (APHA) năm 2025, việc áp dụng phương pháp Pomodoro trực tiếp vào các phiên tự học giúp giảm thiểu đáng kể tình trạng kiệt sức học vụ (academic burnout) và duy trì sự nhất quán cao hơn hẳn so với việc sinh viên tự điều chỉnh thời gian nghỉ ngơi [24].



Hình 4.4: Giao diện bộ đếm giờ Pomodoro và biểu đồ thống kê nỗ lực học tập

4.3 Đánh giá thực nghiệm kiến trúc và thuật toán

Để kiểm chứng tính hiệu quả của sản phẩm, nhóm đã tiến hành chạy thử nghiệm các kịch bản sử dụng thực tế, đặc biệt là đánh giá sự hoạt động của thuật toán Tự động sắp xếp ưu tiên (Auto-Priority) – một trong những điểm nhấn của dự án.

Kết quả thực nghiệm cho thấy thuật toán hoạt động hoàn toàn chính xác và mang lại sự tiện lợi lớn cho người dùng. Khi thử kịch bản tạo một lúc nhiều công việc với các hạn chót (deadline) khác nhau, giao diện lập tức phản hồi đúng theo logic: những bài tập có deadline vào ngày mai hoặc ngày kia sẽ tự động được hệ thống gắn nhãn đỏ (Mức độ Cao) và đẩy lên đầu danh sách. Ngược lại, những bài tập có deadline ở tuần sau sẽ nhận nhãn xanh (Mức độ Thấp) và nằm ở dưới. Thú vị hơn, sau khi để hệ thống chạy qua đêm (nhờ cơ chế Cron Job tích hợp ngầm), sáng hôm sau đăng nhập lại, những

bài tập trước đó ở mức Thấp nhưng nay đã sát ngày nộp bài đều tự động được chuyển sang trạng thái cảnh báo mức độ Cao. Điều này chứng minh ứng dụng hoàn toàn có khả năng đóng vai trò như một "trợ lý ảo", chủ động nhắc nhở tiến độ thay vì chờ người dùng tự vào kiểm tra.

Về mặt hiệu năng và trải nghiệm người dùng, ứng dụng được định hướng thiết kế theo chuẩn PWA (Progressive Web App - Ứng dụng web tiên tiến) và SPA (Single Page Application - Ứng dụng trang đơn). Nhờ đó, thao tác chuyển đổi qua lại giữa các trang như từ Dashboard sang Bảng Kanban hay Trang thống kê diễn ra gần như tức thời, không có hiện tượng chớp trắng trang. Giao diện cũng thể hiện sự linh hoạt (responsive) tốt khi thử nghiệm thu nhỏ cửa sổ trình duyệt hoặc mở bằng điện thoại: các bảng tính, biểu đồ và danh sách công việc tự động xếp chồng lên nhau một cách gọn gàng, nút bấm to rõ, đảm bảo sinh viên có thể theo dõi việc học ở bất cứ đâu.

Đánh giá hiệu năng thời gian thực (Performance Evaluation)

Để đảm bảo hệ thống đáp ứng tốt trải nghiệm của một Ứng dụng đơn trang (SPA), nhóm nghiên cứu đã tiến hành kiểm thử hiệu năng (Performance Testing) đối với các luồng API cốt lõi. Quá trình đo lường thời gian phản hồi (Response Time) được thực hiện thông qua công cụ Postman trong điều kiện mạng tiêu chuẩn. Kết quả trung bình sau 100 lần gọi (requests) cho mỗi API được trình bày trong bảng:

Nhóm API	Phương thức	Endpoint tiêu biểu	Thời gian phản hồi	Đánh giá
Xác thực	POST	/api/v1/auth/login	120 ms	Tốt
Truy xuất	GET	/api/v1/tasks	80 ms	Rất tốt
Khởi tạo	POST	/api/v1/tasks/createTask	150 ms	Tốt
Cập nhật	PUT	/api/v1/tasks/updateStatus	95 ms	Rất tốt

Bảng 4.2: Đánh giá thời gian phản hồi của các API tiêu biểu

Nhận xét: Tất cả các API cốt lõi đều có thời gian phản hồi nằm ở mức dưới 200ms. Mức phản hồi này vượt qua tiêu chuẩn trải nghiệm Web thông thường (dưới 500ms), chứng minh kiến trúc Node.js kết hợp với cơ chế I/O bất đồng bộ (non-blocking) đã hoạt động tối ưu. Tốc độ này đảm bảo thao tác kéo thả thẻ bài tập trên bảng Kanban diễn ra mượt mà, không có độ trễ (latency), mang lại cảm giác phản hồi tức thì cho sinh viên.

4.4 Hạn chế và hướng phát triển trong tương lai

Mặc dù hệ thống đáp ứng tốt các mục tiêu nghiên cứu cốt lõi và cung cấp trải nghiệm Web mượt mà , phiên bản hiện tại vẫn tồn tại một số hạn chế mở ra hướng phát triển cho các nghiên cứu tiếp theo:

Tích hợp hệ sinh thái bên ngoài: Các chức năng phân tích học tập hiện tại mới dừng ở mức trực quan hóa thống kê mô tả (Descriptive Analytics). Hướng

phát triển tiếp theo là áp dụng các mô hình Học máy (Machine Learning) để phân tích hành vi hoàn thành công việc, từ đó đưa ra các gợi ý học tập thông minh (Predictive Analytics) – ví dụ: dự báo khả năng trễ hạn của một đồ án dựa trên dữ liệu thói quen học tập trong quá khứ của sinh viên.

Mở rộng đa nền tảng: Để đáp ứng nhu cầu sử dụng linh hoạt mọi lúc mọi nơi, dự án dự kiến sẽ tối ưu hóa kiến trúc hiện tại thành Progressive Web App (PWA) ở mức độ sâu hơn nhằm hỗ trợ chế độ ngoại tuyến (offline mode) toàn diện, hoặc tái sử dụng mã nguồn logic hiện tại để phát triển phiên bản ứng dụng di động gốc (Native Mobile App) thông qua framework React Native.

4.5 Kết luận và hướng phát triển

4.5.1 Kết luận

Đề tài "Phát triển hệ thống hỗ trợ lập kế hoạch học tập cho sinh viên" (Smart Study Planner) được thực hiện với mục tiêu giải quyết những khó khăn thực tiễn trong việc quản lý thời gian, sự xao nhãng và áp lực từ khối lượng công việc học vụ ngày càng tăng của sinh viên. Trải qua quá trình khảo sát, phân tích yêu cầu, thiết kế kiến trúc và triển khai lập trình, nhóm nghiên cứu đã xây dựng thành công một hệ thái phần mềm khép kín, đáp ứng toàn diện các mục tiêu đã đặt ra ban đầu.

Về mặt công nghệ, hệ thống đã ứng dụng thành công các công nghệ phát triển web hiện đại nhất hiện nay. Kiến trúc Client – Server với sự kết hợp giữa ReactJS ở phía Frontend và Node.js/Express.js ở phía Backend đã mang lại hiệu năng cao và tốc độ phản hồi mượt mà. Việc sử dụng cơ sở dữ liệu quan hệ PostgreSQL thông qua Prisma ORM đảm bảo tính toàn vẹn và bảo mật dữ liệu ở mức tối đa.

Về mặt chức năng và giá trị thực tiễn, hệ thống đã vượt ra khỏi khuôn khổ

của một ứng dụng ghi chú thông thường. Thông qua việc tích hợp các phương pháp quản trị năng suất đã được kiểm chứng khoa học như bảng Kanban và kỹ thuật Pomodoro, hệ thống giúp trực quan hóa luồng công việc và cải thiện đáng kể sự tập trung. Đặc biệt, thành công lớn nhất của đề tài là việc thiết kế và triển khai thuật toán Auto-Priority kết hợp cơ chế chạy ngầm (Cron Job). Cơ chế này hoạt động như một "trợ lý ảo", tự động phân tích chi phí thời gian và cập nhật mức độ khẩn cấp của từng bài tập mỗi ngày, giúp giảm thiểu triệt để sự mệt mỏi khi phải tự ra quyết định của sinh viên.

Nhìn chung, sản phẩm của đề tài đã chứng minh được tính khả thi và mang lại giá trị thực tiễn cao, hỗ trợ sinh viên thay đổi thói quen từ "học tập thụ động, đối phó deadline" sang "chủ động lập kế hoạch và tối ưu hóa thời gian".

4.5.2 Hướng phát triển

Mặc dù đã đạt được những kết quả khả quan và hiện thực hóa được các chức năng cốt lõi, nhóm nghiên cứu nhận thức rõ hệ thống vẫn còn tiềm năng lớn để tiếp tục nâng cấp. Trong giai đoạn tiếp theo, dự án định hướng phát triển theo các trọng tâm sau:

Tích hợp Trí tuệ nhân tạo (AI) và Học máy (Machine Learning): Nâng cấp hệ thống từ mức độ thống kê mô tả (Descriptive Analytics) lên mức độ phân tích dự báo (Predictive Analytics). Dựa trên dữ liệu lịch sử về thời gian hoàn thành các loại bài tập khác nhau của sinh viên, mô hình AI sẽ phân tích thói quen và đưa ra cảnh báo sớm nếu sinh viên có nguy cơ trễ hạn đồ án. Đồng thời, hệ thống sẽ tự động gợi ý chia nhỏ các nhiệm vụ lớn (Breakdown tasks) thành các bước thực hiện tối ưu nhất. Mở rộng kết nối API với hệ sinh thái bên ngoài: Xây dựng tính năng đồng bộ hóa hai chiều (two-way sync) với các nền tảng lịch phổ biến như Google Calendar, Microsoft Outlook và Apple

Calendar. Điều này giúp sinh viên quản lý tập trung mọi lịch trình cá nhân, lịch làm việc bán thời gian và lịch học trên cùng một giao diện duy nhất.

Phát triển phiên bản Ứng dụng di động gốc (Native App): Nhằm tối đa hóa tính tiện dụng và khả năng tương tác mọi lúc mọi nơi, mã nguồn logic hiện tại sẽ được tái sử dụng để phát triển phiên bản ứng dụng trên nền tảng iOS và Android thông qua framework React Native. Ứng dụng di động sẽ hỗ trợ đẩy thông báo nhắc nhở (Push Notifications) trực tiếp (native) đến thiết bị của người dùng khi có công việc sắp đến hạn hoặc đến giờ học.

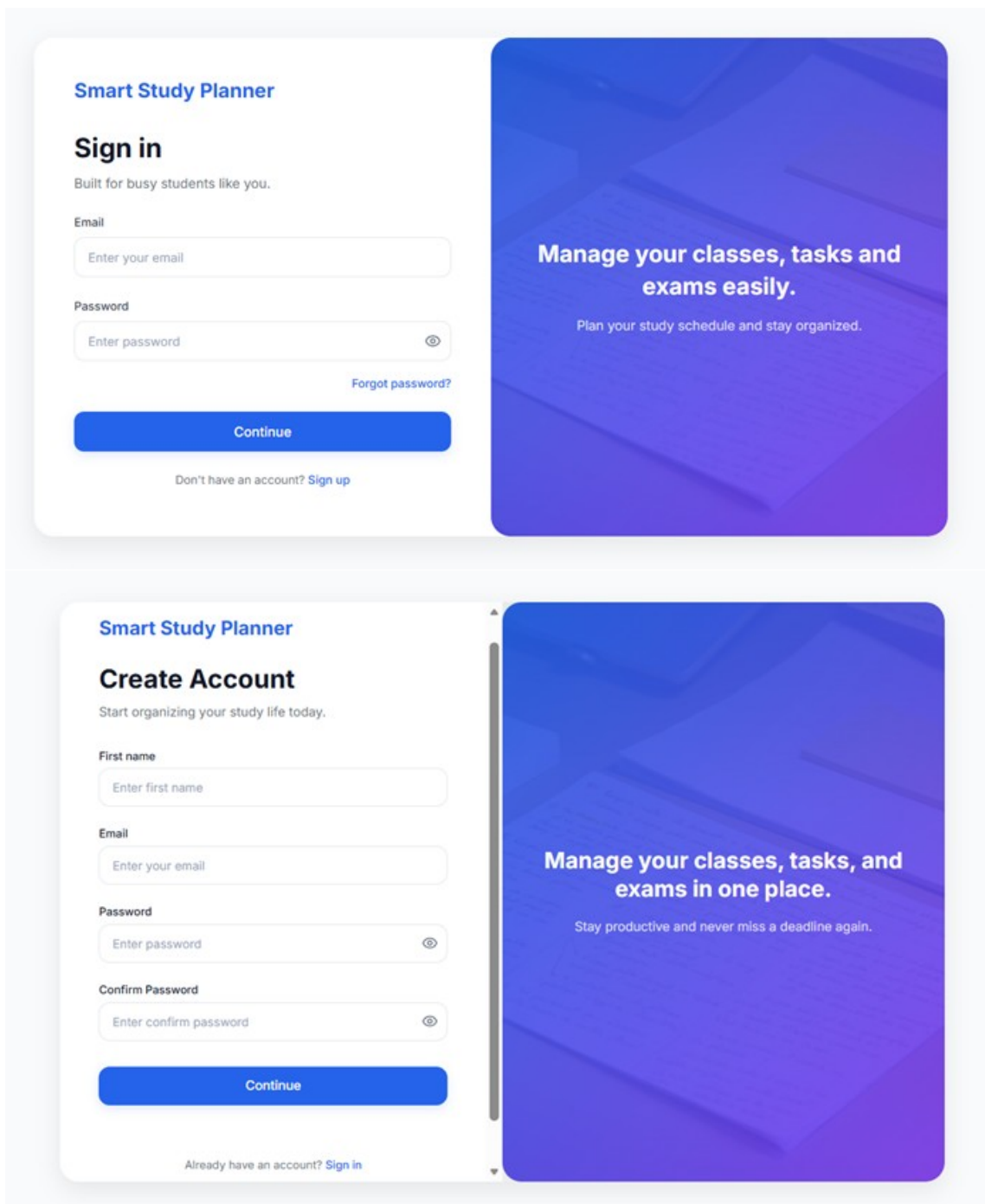
Bổ sung tính năng Học tập nhóm (Collaborative Study): Nâng cấp kiến trúc dữ liệu để cho phép tạo các không gian làm việc nhóm (Group Workspaces). Tính năng này sẽ hỗ trợ sinh viên cùng nhau quản lý các đề án nhóm, phân công thẻ Kanban cho từng thành viên và theo dõi tiến độ đóng góp chung, từ đó mở rộng phạm vi ứng dụng của hệ thống từ cá nhân sang cộng đồng học tập.

HƯỚNG DẪN SỬ DỤNG HỆ THỐNG SMART STUDY PLANNER

1 Khởi tạo và đăng nhập hệ thống

Truy cập: Người dùng truy cập vào đường dẫn của ứng dụng thông qua trình duyệt web. Nhờ tích hợp công nghệ PWA (Progressive Web App), người dùng có thể chọn "Install App" để cài đặt hệ thống trực tiếp lên máy tính hoặc thiết bị di động nhằm sử dụng mượt mà như một ứng dụng gốc (Native App) độc lập.

Đăng ký và Đăng nhập: Người dùng mới cần tạo tài khoản thông qua màn hình Sign Up. Hệ thống yêu cầu xác thực thông tin cơ bản (Họ tên, Email, Mật khẩu) với cơ chế đối chiếu mật khẩu an toàn. Các tài khoản đã tồn tại có thể tiến hành đăng nhập trực tiếp hoặc sử dụng tính năng khôi phục mật khẩu (Forgot Password) nếu cần.



Hình 1.1: Giao diện đăng nhập và cài đặt PWA của hệ thống

2 Hướng dẫn phân hệ các chức năng chính

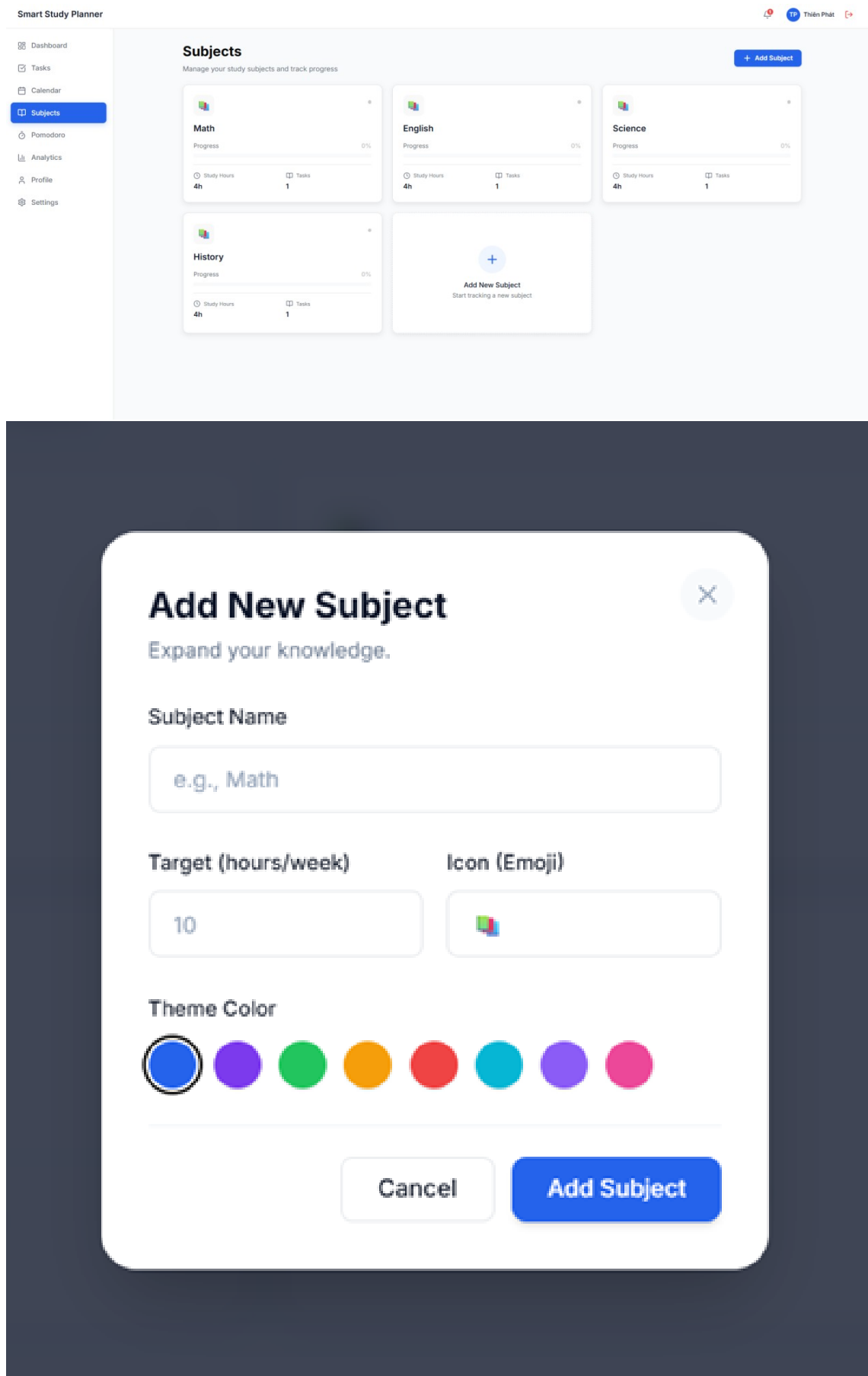
Để khai thác tối đa sức mạnh của hệ thống trong việc quản lý học tập, người dùng được khuyến nghị thực hiện theo luồng logic nghiệp vụ (User Flow) như sau: Tạo Môn học->Xếp Lịch học->Lên Danh sách Công việc->Bắt đầu Học tập.

2.1 Phân hệ quản lý môn học (Subjects)

Đóng vai trò là dữ liệu lõi để phân loại các hoạt động học tập. Thêm mới: Tại thẻ điều hướng Subjects, người dùng nhấn nút "Add Subject". Cửa sổ cấu hình cho phép nhập tên môn học, số giờ học mục tiêu trong tuần (Target hours/week), chọn Icon (Emoji) và chỉ định Màu sắc nhận diện (Theme Color).

Kiểm soát tính toàn vẹn: Hệ thống tích hợp thuật toán kiểm tra trùng lặp thời gian thực. Nếu người dùng nhập trùng tên một môn học đã tồn tại (không phân biệt chữ hoa/thường), hệ thống sẽ từ chối truy xuất và hiển thị thông báo lỗi (Toast Notification) để bảo vệ tính nhất quán của cơ sở dữ liệu.

Theo dõi tiến độ: Mỗi thẻ môn học hoạt động như một widget động, tự động tính toán và hiển thị thanh tiến độ (Progress Bar) dựa trên tỷ lệ phần trăm số lượng Task đã hoàn thành thuộc môn học đó.



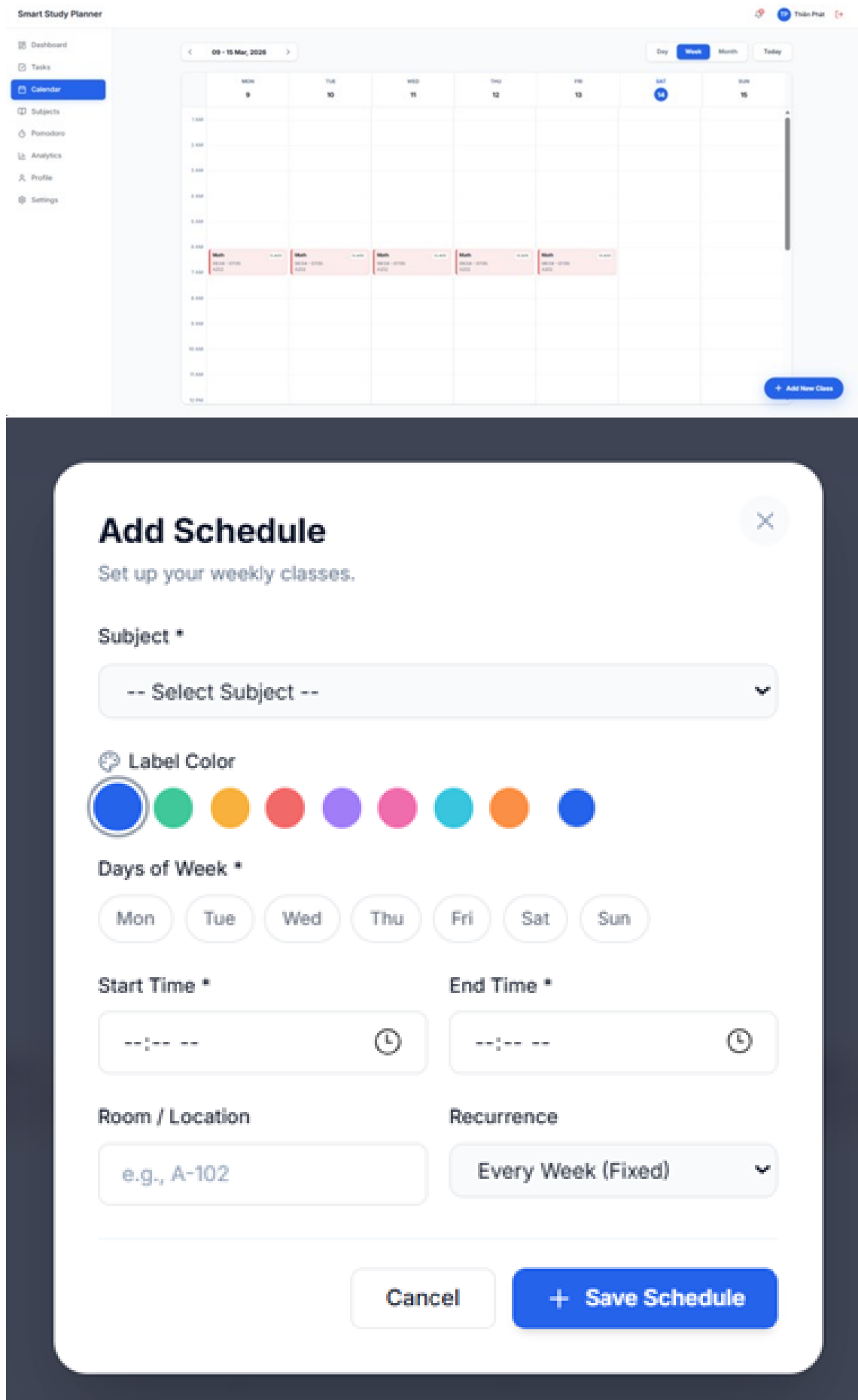
Hình 2.1: Giao diện quản lý môn học và tính năng cảnh báo trùng lặp

2.2 Phân hệ quản lý lịch học (Calendar)

Giao diện hiển thị: Hỗ trợ 3 chế độ xem linh hoạt: Theo Ngày (Day), Theo Tuần (Week) và Theo Tháng (Month). Khối thời gian được tô màu tự động đồng bộ với màu sắc của Môn học đã chọn ở bước 2.1.

Cơ chế lặp (Recurrence): Chức năng được thiết kế đặc thù cho môi trường đại học với 3 tùy chọn thiết lập chu kỳ: Học mỗi tuần (Every Week), Chỉ học tuần chẵn (Even Weeks) và Chỉ học tuần lẻ (Odd Weeks).

Kiểm soát xung đột (Overlap Prevention): Khi lưu lịch học mới, thuật toán sẽ tự động quét đôi chiều. Nếu phát hiện sự chồng chéo thời gian với một lịch học đã tồn tại trong cùng một khung giờ của ngày đó, hệ thống sẽ lập tức chặn hành động và đưa ra cảnh báo nhằm tránh sai sót trong việc đăng ký lịch.



Hình 2.2: Giao diện lưới thời khóa biểu và tính năng chống trùng lịch

2.3 Phân hệ quản lý công việc (Task)

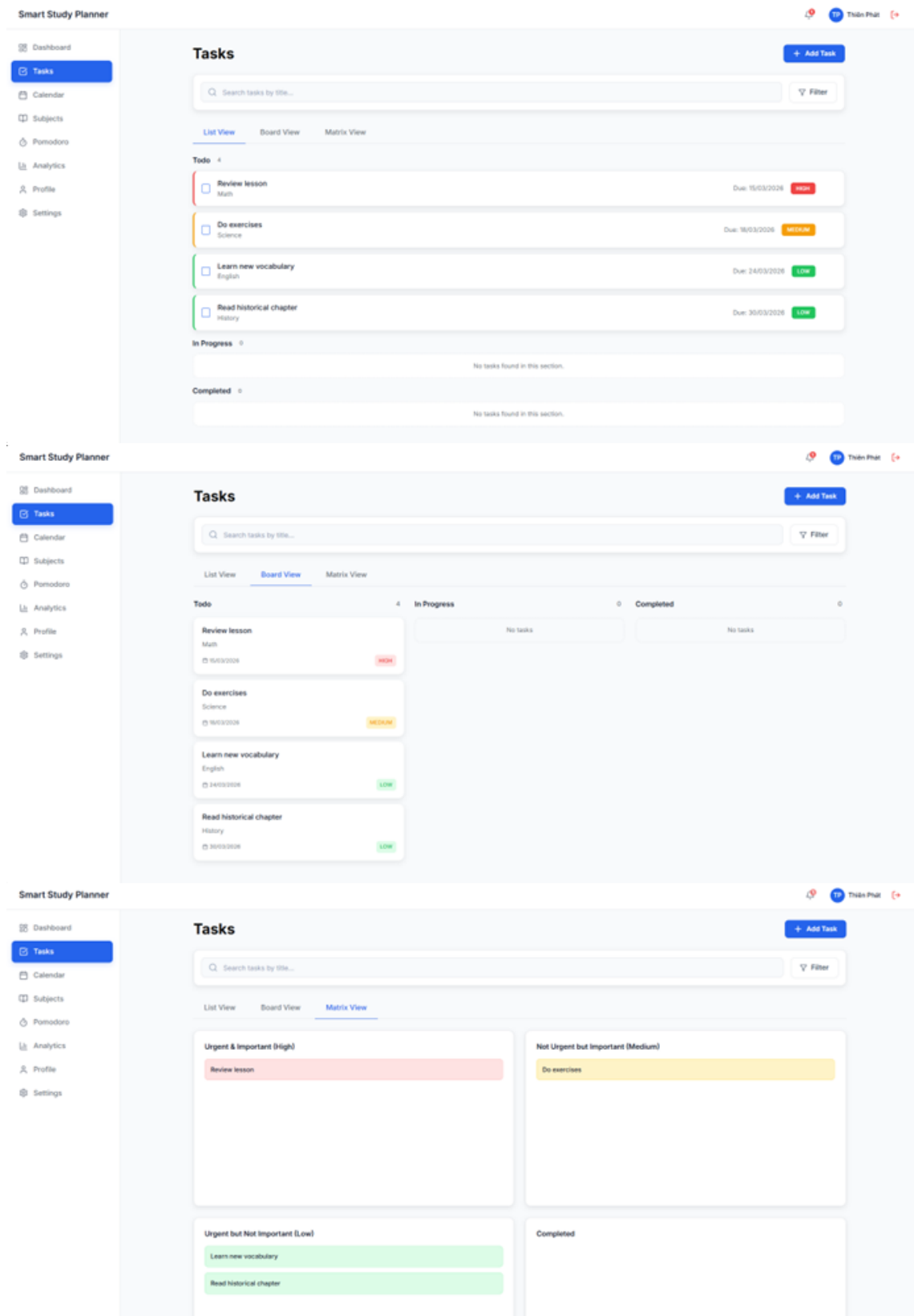
Tạo mới công việc: Cho phép người dùng khởi tạo các đầu việc (Task), gắn kết trực tiếp với một Môn học cụ thể, thiết lập Hạn chót (Due Date) và định định mức độ ưu tiên (Priority: High/Medium/Low).

Đa chế độ giám sát:

List View: Giao diện danh sách trực quan, phân rõ theo nhóm trạng thái. Người dùng có thể nhấp trực tiếp vào Checkbox để thay đổi trạng thái hoàn thành.

Board View: Giao diện bảng Kanban hiện đại, phân tách rõ ràng luồng công việc qua các cột: Todo (Cần làm), In Progress (Đang tiến hành) và Completed (Đã hoàn thành).

Matrix View: Giao diện phân tích dựa trên Ma trận Eisenhower, tự động ánh xạ và phân luồng công việc theo mức độ Khẩn cấp và Quan trọng.



Hình 2.3: Giao diện quản lý công việc đa góc nhìn

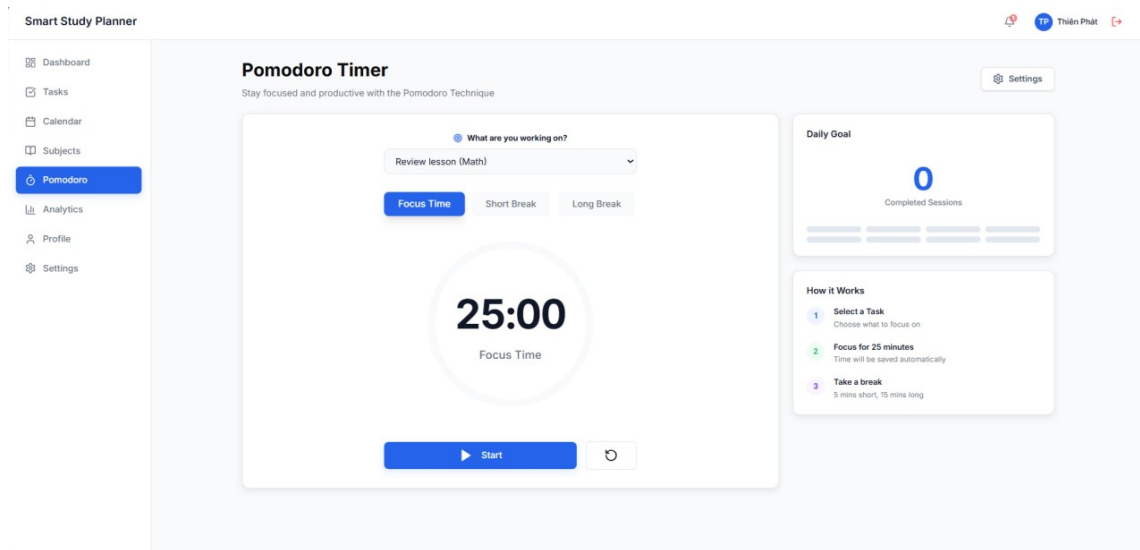
2.4 Phân hệ hỗ trợ tập trung (Pomodoro timer)

Đây là phân hệ thực thi, được liên kết trực tiếp với dữ liệu Công việc (Tasks).

Đồng bộ trạng thái: Trước khi bắt đầu phiên tập trung, người dùng lựa chọn một Task cụ thể từ danh sách thả xuống. Khi nhấn "Start", hệ thống sẽ tự động gọi API chuyển Task đó sang trạng thái In Progress tại phân hệ Tasks.

Chu trình tự động: Đồng hồ đếm ngược hoạt động theo chu kỳ Pomodoro chuẩn. Khi kết thúc một phiên tập trung (mặc định 25 phút), hệ thống tự động ghi nhận thời gian vào cơ sở dữ liệu để phục vụ báo cáo, đồng thời tự động chuyển đổi sang trạng thái nghỉ ngắn (Short Break) hoặc nghỉ dài (Long Break).

Đồng bộ ngầm (Background Sync): Biến đếm thời gian được xử lý ngầm và lưu trữ tại LocalStorage. Nhờ đó, đồng hồ vẫn tiếp tục đếm chính xác ngay cả khi người dùng chuyển đổi qua lại giữa các trang hoặc vô tình tải lại trình duyệt.



Hình 2.4: Đồng hồ đếm ngược Pomodoro tích hợp đồng bộ trạng thái công việc

2.5 Phân hệ hồ sơ và cài đặt (Profile Setting)

Hồ sơ (Profile): Cung cấp các chỉ số đo lường hiệu suất cá nhân (Tổng giờ học, Số Task hoàn thành, Số môn học đang theo dõi). Người dùng có thể tự cấu hình các "Mục tiêu" (Goals) với thanh tiến độ linh hoạt và theo dõi các Thành tựu (Achievements) đã mở khóa.

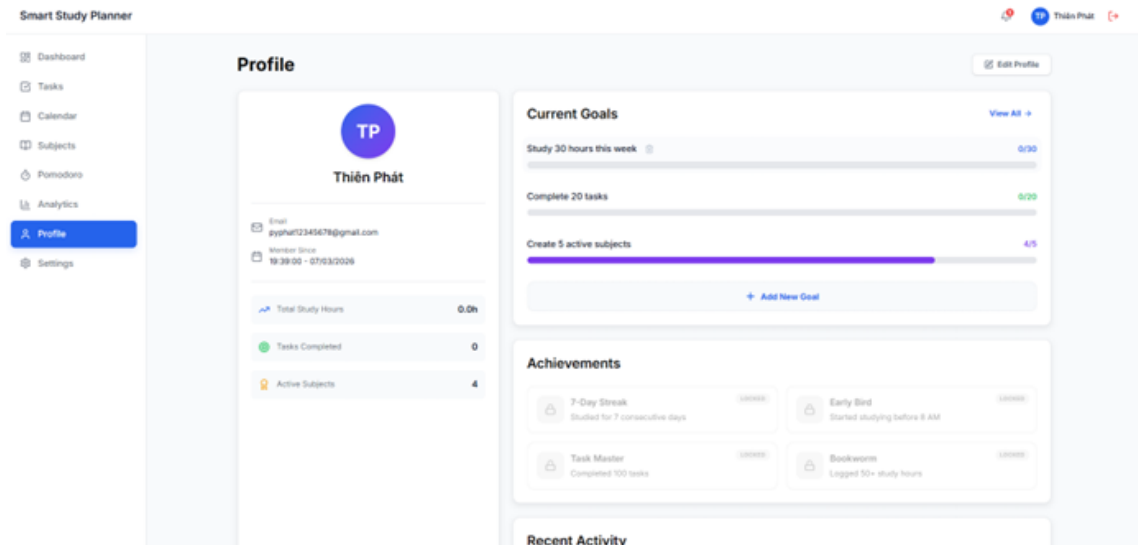
Cài đặt (Settings): Cho phép cá nhân hóa toàn diện trải nghiệm người dùng (UX):

Chuyển đổi tông màu chủ đạo của toàn hệ thống (Primary Color).

Tùy chỉnh kích thước hiển thị văn bản (Font Size).

Chuyển đổi chế độ Giao diện Sáng/Tối (Light/Dark Mode).

Cá nhân hóa các thông số thời gian cho đồng hồ Pomodoro.



Hình 2.5: Giao diện hồ sơ cá nhân và theo dõi tiến độ mục tiêu

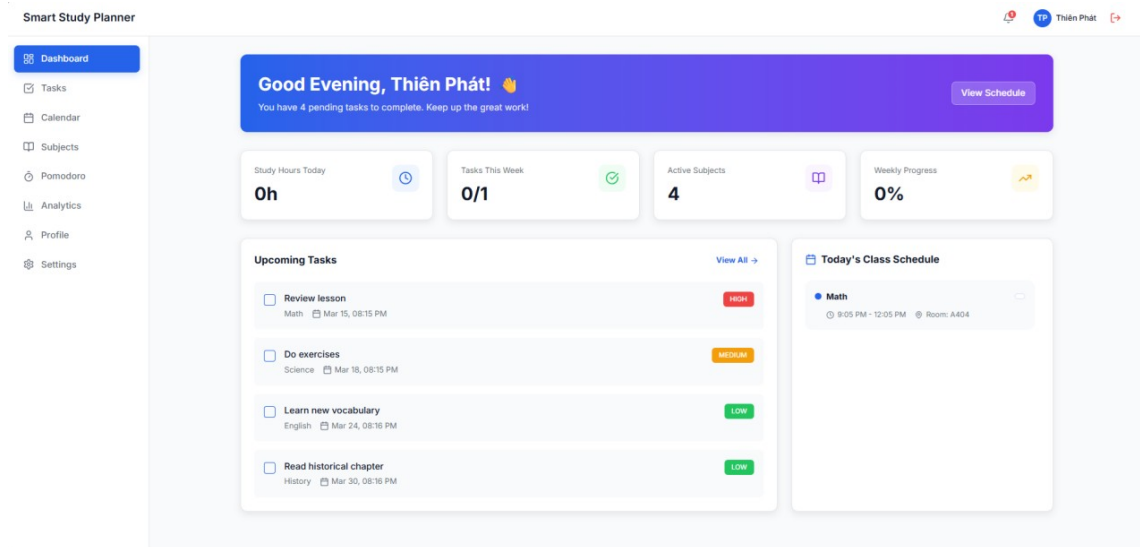
2.6 Bảng điều khiển trung tâm (DashBoard)

Đóng vai trò là Command Center, cung cấp cái nhìn toàn cảnh về tình trạng học tập hiện tại.

Khối thống kê nhanh: Cập nhật thời gian thực về Study Hours, số Tasks và Active Subjects.

Smart Sorting: Phân vùng "Upcoming Tasks" tích hợp thuật toán tự động rà soát, lọc và đẩy các công việc có hạn chót trong vòng 24 giờ tới lên vị trí ưu tiên cao nhất, kèm theo cảnh báo màu sắc.

Hệ thống thông báo (Notification Bell): Trung tâm thông báo góc trên cùng bên phải lưu trữ lịch sử các phiên Pomodoro vừa hoàn thành hoặc đưa ra báo động khẩn cấp về Deadline đang đến gần.



Hình 2.6: Bảng điều khiển trung tâm thể hiện tổng quan tiến độ

TÀI LIỆU THAM KHẢO

- [1] Iffath Ahmed, Grisilda Vidya Bernhardt, and Pooja Shivappa. Prevalence of academic procrastination and its negative impact on students. *Biomedical and Biotechnology Research Journal (BBRJ)*, 7(3):363–370, 2023.
- [2] Chijioke Virgilus Amoke, Moses Onyemaechi Ede, Daphney Mawila-Chauke, and Ifeanyi Emmanuel Obeagu. Disputing and challenging academic procrastination behaviors in students using rebt approach. *Medicine*, 104(9):e41725, 2025.
- [3] Anique BH de Bruin, Eva M Janssen, Julia Waldeyer, and Ferdinand Stebner. Cognitive load and challenges in self-regulation: An introduction and reflection on the topical collection. *Educational psychology review*, 37(3):65, 2025.
- [4] John W Creswell and J David Creswell. *Research design: Qualitative, quantitative, and mixed methods approaches*. Sage publications, 2017.
- [5] Yared Fentaw, Bekalu Tadesse Moges, and Sayed M Ismail. Research article academic procrastination behavior among public university students. 2022.

- [6] Miriam Arnold, Mascha Goldschmitt, and Thomas Rigotti. Dealing with information overload: a comprehensive review. *Frontiers in psychology*, 14:1122200, 2023.
- [7] Nadja Damij and Talib Damij. An approach to optimizing kanban board workflow and shortening the project management plan. *IEEE transactions on engineering management*, 71:13266–13273, 2021.
- [8] Irfan Ahmed, Fatima Munir, Fouzia Nafees, Jabar Mahmood, Shehzad Ashraf Chaudhry, Maryam Javaid, and Khadija Batool. The influence of kanban agile methodology on software project management: A survey method. In *2024 International Conference on Engineering and Emerging Technologies (ICEET)*, pages 1–6. IEEE, 2024.
- [9] Lars Uhlig, Vera Baumgartner, Roman Prem, Katja Siestrup, Christian Korunka, and Bettina Kubicek. A field experiment on the effects of weekly planning behaviour on work engagement, unfinished tasks, rumination, and cognitive flexibility. *Journal of Occupational and Organizational Psychology*, 96(3):575–598, 2023.
- [10] Eva JC Smits, Niklas Wenzel, and Anique de Bruin. Investigating the effectiveness of self-regulated, pomodoro, and flowtime break-taking techniques among students. *Behavioral Sciences*, 15(7):861, 2025.
- [11] Felicitas Biwer, Wisnu Wiradhany, Mirjam GA Oude Egbrink, and Anique BH De Bruin. Understanding effort regulation: Comparing ‘pomodoro’breaks and self-regulated breaks.
- [12] Kristina Wu. Navigating the balance between urgency and importance: Exploring task prioritization strategies across environments. 2024.

- [13] Bryan Nagib Zambrano Manzur, Fabián Andrés Espinoza Bazán, Yamilis Fernandez, and Carlos Cruz Corona. Solving the interdependence of weighted shortest job first variables by applying fuzzy cognitive mapping. *Information*, 16(11):944, 2025.
- [14] Antonio Ragusa, Jeronimo González-Bernal, Ruben Trigueros, Valeria Caggiano, Noelia Navarro, Luis A Minguez-Minguez, Ana I Obregón, and Carmen Fernandez-Ortega. Effects of academic self-regulation on procrastination, academic stress and anxiety, resilience and academic performance in a sample of spanish secondary school students. *Frontiers in psychology*, 14:1073529, 2023.
- [15] Sabina Valente, Sergio Dominguez-Lara, and Abílio Lourenço. Planning time management in school activities and relation to procrastination: A study for educational sustainability. *Sustainability*, 16(16):6883, 2024.
- [16] Florian Pinggera. Efficient personnel task scheduling and student assignment for sports schools—practical implementation using a model and a heuristic based approach/submitted by florian pinggera, bsc. 2023.
- [17] Salihu Ibrahim Rabi. Studyflow: Enhancing academic productivity through ai-driven task prioritization and personalized scheduling. 2025.
- [18] Zhisheng Xiong. Design and implementation of student management system based on b/s. *information management*, 3:4, 2023.
- [19] AA Prayogi, M Niswar, Indrabayu, and M Rijal. Design and implementation of rest api for academic information system. In *IOP Conference Series: Materials Science and Engineering*, volume 875, page 012047. IOP Publishing, 2020.

- [20] Nnaemeka Kingsley Ugwumba and Peter Sunday Jaja. Enhanced task prioritization system using deep-q-network model. 2025.
- [21] Muhammad Usman Sana and Zhanli Li. Efficiency aware scheduling techniques in cloud computing: a descriptive literature review. *PeerJ Computer Science*, 7:e509, 2021.
- [22] Neeta Diwan. Optimising performance in react apps. 2024.
- [23] Viljan Mahnič. Kanban in software engineering education: an outline of the literature. *World Transactions on Engineering and Technology Education*, 17(1):23–28, 2019.
- [24] Sana Kausar Habiya and Jaweriya Azeem. Role of the pomodoro technique in reducing stress and preventing burnout among college students with a focus group on neurodivergent. In *APHA 2025 Annual Meeting and Expo*. APHA, 2025.